PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines

A PROJECT REPORT

Submitted by

DARSHAN GOWDA S - 20221IST0055

CHIRAG GOWDA S V - 20221IST0112

CHETHAN C - 20221IST0069

Under the guidance of,

Dr. Afroz Pasha

BACHELOR OF TECHNOLOGY

IN

INFORMATION SCIENCE AND TECHNOLOGY

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "**Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines**" is a bonafide work of "Darshan Gowda S (20221IST0055), Chirag Gowda S V (20221IST0112), Chethan C (20221IST0069)", who have successfully carried out the project work and submitted the report for partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in INFORMATION SCIENCE AND TECHNOLOGY during 2025-26.

Dr. Afroz Pasha
Project Guide
PSCS
Presidency University

Ms. Benitha Christinal J
Program Project Coordinator
PSCS
Presidency University

Dr. Sampath A K
School Project Coordinators
PSCS
Presidency University

Dr. Pallavi R
Head of the Department PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS
Presidency University

Name and Signature of the Examiners

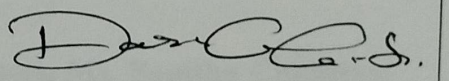
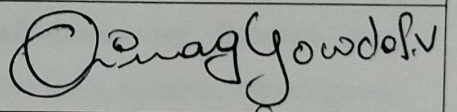
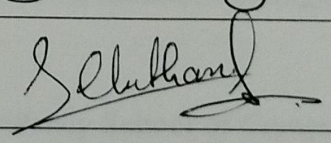
1)	Dr. Afroz Pasha	24/4/26
2)	Ms. Imba B.H	24/4/26

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in INFORMATION SCIENCE AND TECHNOLOGY, at Presidency University, Bengaluru, named Darshan Gowda S, Chirag Gowda S V, Chethan C, hereby declare that the project work titled "**Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines**" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in INFORMATION SCIENCE AND TECHNOLOGY, during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Darshan Gowda S	20221IST0055	
Chirag Gowda S V	20221IST0112	
Chethan C	20221IST0069	

PLACE: BENGALURU

DATE: 24/04/2026

ACKNOWLEDGEMENT

For completing this project work, we have received the support and the guidance from many people whom we would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

We would like to sincerely thank our internal guide **Dr. Afroz Pasha, Assistant Professor—Senior Scale, Presidency School of Computer Science and Engineering,** Presidency University, for his moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

We are also thankful to **Dr. Pallavi.R, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for her mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N, Dean PSCS & PSIS, Presidency School of Computer Science and Engineering** and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of our project work.

We are grateful to **Dr. Sampath A K, PSCS** Project Coordinators**, Ms. Benitha Christinal J, Program Project Coordinator**, **Presidency School of Computer Science and Engineering**, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

DARSHAN GOWDA S
CHIRAG GOWDA S V
CHETHAN C

ABSTRACT

Due to harsh thermal and mechanical stress, aircraft turbofan engines degrade over time and appropriate Remaining Useful Life (RUL) prediction is necessary to safe, cost-effective Predictive Maintenance (PdM). Recent approaches are based on either deep learning structures that capture the temporal dynamics of degradation patterns, or classical machine learning approaches that makes use of statistical distributions of features without leveraging the complimentary advantages of the two. It is postulated in this project that a Hybrid XGBoost-LSTM Ensemble with Soft Attention Mechanism can be used to predict turbofan engine RUL, which are evaluated on all the four sub-data sets of the NASA C-MAPSS benchmark.

Two parallel branches are used in the framework. The former is an augmented with attention LSTM network that models multivariate sensor time-series as 50-cycle sequences in a 73-dimensional feature space containing raw operating condition settings, 14 variance-filtered sensor channels, and 56 rolling window statistical features. The second one is an XGBoost gradient boosted tree regressor which uses the same 73-dimensional feature vector per cycle. The soft attention mechanism learns importance weights of the 50-cycle window, which allows the LSTM to pay attention to degradation-phase cycles. Huber Loss with $\delta = 10$ guarantees ability to withstand outlier engine paths. In the case of multi-condition sub-data sets FD002 and FD004, KMeans clustering with k=6 can be used to identify predominant flight regimes and applies per-cluster Z-score normalization to isolate sensor baseline shifts of true degradation indicators. This last prediction combines the two branches using a per-dataset learned weight α^*, which is optimized with grid search and then the result is converted to a three-tier maintenance status: Critical, Warning, or Healthy.

On single-condition sub-datasets FD001 and FD003, the standalone LSTM has the highest performance with RMSE of 15.58 and 13.54 and R^2 of 0.85 and 0.88 respectively. The Hybrid Ensemble performs better than the two individual models on multi-condition sub-datasets FD002 and FD004 with RMSE of 17.47 ($R^2 = 0.83$) and 23.56 ($R^2 = 0.70$), respectively, supporting the argument that the two modelling paradigms provide complementary information under multi-condition operating complexity. This framework adds the principled cross-paradigm ensemble blending, KMeans-based normalization, soft attention, Huber Loss training, and a maintenance status classification layer between numerical RUL prediction and maintenance action taking.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

Abbreviation	Full Form
ACM	Association for Computing Machinery
AI	Artificial Intelligence
Adam	Adaptive Moment Estimation (optimizer)
BatchNorm	Batch Normalization
C-MAPSS	Commercial Modular Aero-Propulsion System Simulation
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
EWMA	Exponentially Weighted Moving Average
FC	Fully Connected (layer)
FD001	C-MAPSS Sub-dataset 1 (1 operating condition, 1 fault mode)
FD002	C-MAPSS Sub-dataset 2 (6 operating conditions, 1 fault mode)
FD003	C-MAPSS Sub-dataset 3 (1 operating condition, 2 fault modes)
FD004	C-MAPSS Sub-dataset 4 (6 operating conditions, 2 fault modes)
GB	Gigabytes
GPU	Graphics Processing Unit
HPC	High-Pressure Compressor
HPT	High-Pressure Turbine
ICPHM	IEEE International Conference on Prognostics and Health Management
ICLR	International Conference on Learning Representations
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
JSON	JavaScript Object Notation
KMeans	K-Means Clustering Algorithm
L1	L1 Regularization (Lasso)
L2	L2 Regularization (Ridge)

LPC	Low-Pressure Compressor
LPT	Low-Pressure Turbine
LR	Learning Rate
LSTM	Long Short-Term Memory
M1	Module 1 — Data Preparation and Encoding
M2	Module 2 — Sequence Construction and Windowing
M3	Module 3 — Model Architecture and Training
MAE	Mean Absolute Error
MinMax	Min-Max Normalization / MinMaxScaler
ML	Machine Learning
MRO	Maintenance, Repair and Overhaul
NASA	National Aeronautics and Space Administration
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
PdM	Predictive Maintenance
PHM	Prognostics and Health Management
PRONOSTIA	Prognostics and Health Management Bearing Dataset Repository
PyTorch	Open-source Deep Learning Framework (by Meta AI)
(R^2)	Coefficient of Determination
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
RUL	Remaining Useful Life
SDG	Sustainable Development Goal (United Nations)
SEQ_LEN	Sequence Length (sliding window size = 50 cycles)
SHAP	SHapley Additive exPlanations
SIGKDD	ACM Special Interest Group on Knowledge Discovery and Data Mining
SVM	Support Vector Machine
SVR	Support Vector Regression
TensorRT	NVIDIA Tensor Runtime (inference optimization library)

UN	United Nations
URL	Uniform Resource Locator
VRAM	Video Random Access Memory
XGBoost	Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Safety and reliability of any engineering sector in the world are some of the highest to aviation industry. The core of any commercial airliner is the turbofan engine - an extraordinarily impressive work of mechanical engineering which turns fuel into thrust in conditions of high thermal and mechanical stress, cycle after cycle, flight after flight. With time, this continuous operations pressure slowly wears out important internal parts especially the High-Pressure Compressor (HPC) stages and fan blades. When this degradation is not monitored in other cases, it results in the gradual performance decline, component breakdowns, and the worst scenario is loss of life and planes in disastrous scenarios [8].

The financial aspect of engine upkeep is also urgent. The biggest one cost in the global aviation operations is the maintenance, Repair and Overhaul (MRO) spending on engines. Airlines that do not maintain their aircraft on time run the risk of unplanned groundings and failure events, whilst airlines that maintain their aircraft on a schedule that is too slow waste colossal resources in overhauling components that still had a lot of useful life left in them. Both are not operationally acceptable and financially acceptable in an industry where there is a slim margin and no compromise of safety.

Predictive Maintenance (PdM) is the solution to this dilemma. Instead of responding to failures, or servicing on a fixed and programmed schedule, PdM uses the flow of sensor data that is continuously generated by engines under operation to estimate the amount of operational life each individual engine has left - its Remaining Useful Life (RUL). It is possible to schedule the maintenance at an optimum time with an accurate prediction of the RUL: not too late to use the part fully, but not too soon to take the risk of failure [2], [6]. The safety implications of such capability and cost reduction are far-reaching, as are the implications of availability of its fleet and overall operations efficiency.

When a multivariate time-series of sensor measurements is known up to the current cycle then the RUL estimation is mathematically a regression problem, and the model must then predict the number of additional operational cycles in which the engine will operate before it

reaches a pre-defined failure threshold. It is much more difficult, though, than an ordinary regression problem. The engines respond in numerous flight regimes (varying altitude, speed, throttle settings) and sensor baselines respond to these variations in a superficially similar way to the degradation signals [10]. Atmospheric variability and measurement error also contribute to noise added to the raw measurements of sensors. In addition, full run-to-failure paths on which to train models are also inherently rare, and generalization may be problematic.

The machine learning community has achieved a lot progress on this problem in the last 10 years. The earliest attempts with Random Forests and Support Vector Regression revealed that competition RUL estimates could be created using a carefully designed statistical feature [1], [3]. One of the biggest advances was in the creation of Long Short-Term Memory (LSTM) networks, which through their recurrent mechanisms could learn complex temporal degradation dynamics directly to a raw sensor input without having to engineer expensive features [12], [14]. Subsequent works suggested hybrid CNN-LSTM models and attention models, which have since been enhanced in terms of accuracy [15], [17], and [18]. Meanwhile, tree models such as XGBoost demonstrated that gradient boosting methods might be competitive on tabular feature representations of the same data [11].

These developments have created a gap that has remained in the literature. The vast majority of current methods are dedicated to one modelling paradigm: either a deep learning model that learns temporal degradation dynamics, or a classical machine learning model that uses statistical distributions of features. The two paradigms are in a very real sense complementary with recurrent networks being most effective at sequential dynamics, and tree-based models being most effective at exploiting the statistical structure of the feature space but principled ensemble strategies that combine the two in a data-driven, adaptive fashion being less well developed. More so, the issue of the multi-condition engine operation is complicated by the necessity of the complex normalization strategies that are not well or not discussed in much of the published literature [3], [5].

This would be the direct point of gap filling in this project where a Hybrid XGBoost-LSTM Ensemble model with a soft Attention Mechanism is proposed to predict the turbofan engines RUL. On all four of its sub-datasets of increasing complexity, the system is tested against the NASA C-MAPSS benchmark. The framework suggests four broad technical innovations, namely, a learned, per-dataset combining weight α of which the adaptive combination of

LSTM and XGBoost branches prediction is possible; a KMeans-based operating condition normalization approach that allows flight regime variation and true degradation to be decoupled; a soft attention mechanism that allows the LSTM to pay attention to the most informative cycles during the degradation phase; and Huber Loss training for outlier robustness.

1.2 SCOPE AND LIMITATIONS

The research problem that will define the scope of this project is the data-driven Remaining Useful Life prediction on aircraft turbofan engines in both single-condition and multi-condition operating conditions. The system has been developed and tested on the NASA C-MAPSS dataset, which is a four-sub-dataset (FD001, FD002, FD003, FD004) model of a two-spool, high-bypass turbofan engine in different fault modes and complexities of flight regimes only [10]. The scope of the modelling includes the entire machine learning pipeline: the raw sensor data Preprocessing and the operating condition normalization, the feature engineering and model training, and the ensemble optimization and evaluation.

The project is scaled to a regression environment with supervision in which full run-to-failure paths can be used to train and point estimates of RUL are the main prediction object. The architecture consists of two complementary modelling branches an attention-augmented LSTM to model temporal sequences and an XGBoost regressor to exploit statistical features, which are integrated by a learned weighted ensemble. The scope also encompasses designing and executing a per-dataset blending weight optimization algorithm based on grid search on a validation partition held out.

A number of significant constraints should be mentioned. First, the system is tested on simulated data only produced by the C-MAPSS simulation environment. Although C-MAPSS is the factual standard of turbofan prognostics study, it might not be in full agreement with the intricacy and sound nature of genuine flight recorder data. It is still unclear whether or not generalization to real engine data, e.g. data found in the PHM 2008 or PRONOSTIA datasets can be done. Second, the blending weight α is optimized on a held out partition of the same dataset distribution; in genuinely novel operating conditions the weight might fail to be generalized without re-optimization. Third, the existing structure generates point estimates of RUL that are not accompanied by uncertainty quantification. Prediction intervals and estimates of confidence are extremely desirable in safety-critical maintenance decision-

making, and lack of them is one of the weaknesses of the existing system. Fourth, the training resources needed such as a Tesla T4 GPU with 15.6 GB VRAM might not be easily accessible in the resource-limited industrial deployment setting. Lastly, the RUL capping strategy used (capping 125 cycles) is a modelling assumption, which, although typical in the literature, may not have the same degradation onset properties of all real engine populations.

1.3 PROBLEM STATEMENT

Timely and precise estimation of Remaining Useful Life (RUL) of aircraft turbofan engines using multivariate sensor time-series data is an important unresolved issue in aerospace prognostics and health management. Even with the high level of research, the following specific issues are still present in the body of literature:

Limitations of single-paradigm modelling: Current state-of-the-art AIs are either deeply learned (with LSTM networks and variants), or trained using classical machine learning algorithms like gradient boosting and random forests. Deep learning models are suitable to model temporal degradation dynamics of sequential data, but are not so well able to exploit the statistical distribution of the feature space. On the other hand, tree-based models can do very well with feature-level statistical reasoning but are necessarily not able to model sequential temporal dependencies. There is no established principled, adaptive ensemble strategy that lines both the paradigms with a data-driven blending weight optimized on a case-by-case basis [3], [4].

Multi-condition normalization: Under operating conditions where engines are exposed to a range of different flight regimes (the FD002 and FD004 sub-data of C-MAPSS) the raw sensor readings are subject to systematic changes in baseline due to variation in flight conditions, and not due to component degradation. In the absence of a suitable normalization strategy, both paradigms of modelling can confound condition induced sensor variation with actual degradation signals, introducing systematic prediction errors. Most published methods either do not consider multi-condition settings or consider them by methods of normalization that are not principled enough [5], [10].

Mechanism Attention mechanism under-exploration in prognostics: Soft attention mechanisms, which have achieved transformative effect in the fields of natural language processing and sequence-to-sequence tasks [13], have not been well studied in the context of

industrial prognostics. The selective attention weighting of informative time steps, especially the critical degradation stage of an engine lifecycle is an unexploited capability of enhancing the accuracy of RUL prediction.

Absence of actionable linkage of maintenance decisions: The vast majority of published RUL prediction systems provide accuracy measures without providing operational sable maintenance decision structures of model outputs. The disconnect between a numerical estimate of RUL and a practical decision on maintenance scheduling, such as risk classification and assigning maintenance priority is still widely unexplored [6], [7], [9].

Overall, the essence of the problem that this project will solve is as follows: How can the complementary capabilities of temporal sequence modelling (LSTM with attention) and statistical feature modelling (XGBoost) be adapted together in a principled, per-dataset ensemble approach, with additional assistance provided by operating condition-aware normalization, to generate higher-quality RUL prediction accuracy across turbofan engines operating in both single-condition and multi-condition operating environments.

1.4 MOTIVATION

The rationale behind this project is the practical urgency of the aviation maintenance issue, as well as the theoretical possibility of the gaps in the literature on prognostics.

In a practical sense, the implications of wrongful prediction of RUL are dire and skewed. Underestimating the remaining life, resulting in an over-prediction of RUL - projecting more life than there is available, will probably cause the components to fail in the middle of the operation, and the effects of this are potentially disastrous. A false forecast wastes maintenance resources and lowers the availability of aircraft. This issue is of great economic magnitude: engine MRO is the biggest single item in airline operating expenses world-wide and even a slight increase in the accuracy of maintenance schedules results in significant financial savings and safety gains on the fleet level [8]. The growing use of health monitoring systems and digital twins by the aviation industry forms the data infrastructure and the business motivation to implement advanced predictive algorithms.

In the research perspective, the motivation is based on an evident theoretical gap. The two most popular machine learning models used in the RUL prediction recurrent neural networks and gradient boosting methods represent two very different sides of the data: temporal

dynamics and statistical feature distributions, respectively. The instinct that these two perspectives are complementary is sound, but the literature has not created a rigorous, adaptive ensemble framework, which integrates them of a principled manner [19]. The grid-search Optimization of a per-dataset blending weight on a held-out validation partition provides a data-driven procedure to ascertain the optimal amount of contribution of each modelling paradigm, as opposed to heuristic combination principles.

Besides, the problem of multi-condition functioning is a realistic and underestimated complication. The engine of real aircrafts does not work in one steady-state condition; they vary over the scope of altitude, speed, and throttle positions in every flight cycle. The rationale behind the KMeans-based clustering and per-cluster normalization approach that is used in this project is the necessity to separate the real component degradation signal and the spurious sensor variation signal caused by a flight regime transition, which is crucial to sound prognostics in practical deployment scenarios [10].

Lastly, the rationale behind the desire to use soft attention is based on the fact that not every time step in the history of engine operation is equally informative to use in RUL estimation. The initial, healthy period of engine operation has fewer prognostic clues as compared with the degradation period, and a model which learns to concentrate its representational capacity on the latter would be projected to give more correct and accurate predictions [13], [17], [18]. The effectiveness of the attention mechanisms in other areas of sequential prediction gives good preconditions to their use in prognostics of turbofans.

1.5 METHODOLOGY OVERVIEW

The proposed system is based on a systematic, end-to-end machine learning pipeline that is aimed at dealing with every of the challenges identified in a principled and modular way. The system architecture is described in Chapter 3 (Figure 3.1), and the following is the high-level overview of the most important elements of methodology.

Data Acquisition and Preprocessing: NASA C-MAPSS dataset serves as the experimental testbed, which consists of four sub-datasets (FD001-FD004) of different sizes, i.e., in terms of the number of training engines, test engines, operating conditions, and fault modes [10]. Raw sensor data are then filtered with a variance-based filtering mechanism to remove seven effectively constant sensor channels (s1, s5, s6, s10, s16, s18, s19) and leave 14 active

sensors with significantly different degradation characteristics. Normalization of operating conditions is also used differentially, with a global MinMax scale used in single-condition sub-datasets (FD001, FD003) and KMeans clustering (k = 6) used in multi-condition sub-datasets (FD002, FD004) to identify the six dominating flight regimes and normalize each condition per-cluster using a Z-score, RUL labeling is limited to 125 cycles to target the informative part of the engine lifecycle.

Feature Engineering: Each engine cycle is represented by a 73-dimensional feature vector consisting of three raw settings of the operating conditions, 14 variance-filtered sensor values, and 56 engineered features based on four 10-cycle rolling window statistics per sensor of the rolling mean, rolling standard deviation, linear slope, and exponentially weighted moving average (EWMA with $\alpha = 0.1$). This feature space supports the LSTM branch (a time-ordered sequence) as well as the XGBoost branch (a tabular snapshot).

LSTM and Attention Mechanism: This sequential modelling branch takes input tensors of size [Batch × 50 × 73] and uses two stacked layers of LSTM (hidden dimension = 128) to process the input with dropout regularization ($p = 0.2$) after the first layer [12]. Soft attention is applied to the output of the second LSTM layer, which takes the form of a weighted context vector, which highlights the most informative cycles in the sequence window [13]. The context vector is sent through a fully connected head to generate the LSTM branch RUL prediction. The training model is Huber Loss with $\delta = 10$ [16] where the small error is penalized quadratically, and the large error is penalized with robust linear penalty, reducing the impact of the outlier engine paths update on the gradient.

XGBoost Regressor: This branch of tabular modelling uses an XGBoost gradient boosted tree ensemble which is fitted on the 73-dimensional feature vector [11]. The number of trees that will be used is 1,500 in single-condition sub-datasets and 2,000 in multi-condition sub-datasets, learning rate is 0.03, maximum depth is 3, and L1/L2 regularization ($\lambda_1 = 1.0$, $\lambda_2 = 5.0$). Early termination is implemented on the basis of holdout RMSE.

Hybrid Ensemble and Alpha Optimization: Final RUL prediction is the weighted linear combination of the two output of the branches: $\hat{y}\_hybrid = \alpha \cdot \hat{y}\_XGB + (1 - \alpha) \cdot \hat{y}\_LSTM$. Optimization of the blending weight α is done on each sub-dataset by grid search on the interval [0, 1] with a step size of 0.01, evaluated on a held-out validation partition. The

optimal α* values obtained thereof are a reflection of the relative contribution of each modelling paradigm to each level of complexity of operating condition.

Assessment: The three complementary measures (Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R^2) are used to evaluate the framework on the four C-MAPSS sub-datasets. Each sub-dataset has all three variants of the model to be reported as standalone LSTM, standalone XGBoost, and Hybrid Ensemble so that the comparison could be made transparently.

1.6 ORGANISATION OF THE THESIS

The project report is arranged in a way, which allows the reader to follow the reasoning in a coherent way, through the inspiring background and the theoretical premises, to the technical application, the experimental testing and the extended implications of the offered hybrid RUL prediction framework. The subsections below outline the logical flow of the research work, how the contributions have been mapped across the chapters, how the project conforms to the United Nations Sustainable Development Goals, and a summary of each chapter.

1.6.1 Logical Flow of the Research Work

The study is logically organized in three broad steps namely problem identification and contextualization, solution design and implementation, and evaluation and reflection. The initial stage that covers Chapters 1 and 2 develops the context of the problem. Chapter 1 places the issue of RUL prediction into the greater context of the operations and economics of the aviation maintenance industry, recognizes the gaps in current literature, and states the precise research issues and interest that lead to the present proposed method.

Chapter 2 critically reviews the literature on the topic, and surveys the development of prognostic techniques, both classical machine learning methods and deep learning and hybrid methods, and exactly defines what gap the proposed system will fill. The second stage, which falls within Chapters 3 and 4, is the solution design and implementation. In chapter 3, the entire research methodology, the proposed architecture, data Preprocessing strategy and the overall workflow of the hybrid framework are described.

Chapter 4 records the implementation details, module-by-module, and the algorithms, system components integration, and the simulation outputs produced in the course of training and testing.

Chapters 5 and 6 deal with the third phase, which is evaluation and reflection. Chapter 5 displays quantitative experimental findings and analytical observations, comparing all model variations across all sub-datasets. Chapter 6 summarizes the main contributions and results, talks about the study limitations and specifies the tangible paths of future research.

1.6.2 Contribution Mapping Across Chapters

The four major technical contributions of this project are spread all over the report in the following manner. Conceptually Chapter 1 (Sections 1.3 and 1.4), formally defined in Chapter 3 (Proposed Architecture and Workflow), and implemented and documented in Chapter 4 (Module-wise Implementation), the hybrid LSTM-XGBoost ensemble approach with per-dataset learned blending weight α are presented and evaluated experimentally in Chapter 5 (Comparative Results and Analysis).

The operating condition normalization strategy provided in Section 1.3 is inspired by the KMeans algorithm, its construction is described in Chapter 3 (Data Collection and Preprocessing), and its effect measured in comparison in Chapter 5. Soft attention mechanism of LSTM is described in Section 1.5, its architectural specification is discussed in Chapter 3, its implementation is discussed in Chapter 4, and its performance impact is discussed in Chapter 5. Chapter 3 provides the description of the Huber Loss training procedure and Chapter 4 presents the training curves and screenshots of the convergence behaviour which are analyzed in Chapter 5.

1.6.3 Sustainable Development Goals (SDGs)

The suggested hybrid RUL prediction framework has a significant meaning to the various goals of the United Nations Sustainable Development Goals (SDGs), as shown in Figure 1.1. The goals of the project are evaluated in relation to the SDG framework to indicate the further relevance of intelligent predictive maintenance development in the aviation industry to the society and environment on the whole [23].

SDG 9 — Industry, Innovation, and Infrastructure: The project has a direct contribution to SDG 9 because it creates innovative machine-learning infrastructure in the aviation maintenance industry. The hybrid ensemble framework is a technical innovation to the industrial prognostics field, which aligns with the SDG 9 requirement to develop resilient infrastructure, inclusive and sustainable industrialization, and innovation. The system helps to create more intelligent, data-driven maintenance infrastructure that will decrease unplanned downtime and enhance the reliability of air transportation systems by making it easier to predict the degradation of the engine.

SDG 8 — Decent Work and Economic Growth: Accurate RUL prediction is a direct contribution to SDG 8, minimizing the wastage of maintenance resources and unnecessary overhaul processes, and eliminating the need less expensive unplanned engine failures. These deliverables are related to the economic sustainability of airline operation and the aviation MRO sector, which sustains millions of jobs in the world. The project will help to achieve sustainable economic growth in the aviation industry by cutting down on the unnecessary spending and enhancing efficiency in its operations.

SDG 13 — Climate Action: Aviation is a major source of greenhouse gases emissions in the world. The proposed system is expected to decrease the rate of early part replacement and the manufacturing energy costs by prolonging the engine component life cycle by ensuring that the maintenance interventions are performed at the optimal time to ensure the engine components last longer. Moreover, engines in good condition have a higher thermodynamic efficiency, that is, they use less fuel per flight cycle and thus produce less carbon. This is in line with the SDG objective number 13 of taking urgent measures to reduce climate change and its effects.

SDG 3 — Good Health and Well-Being: Aviation safety has direct relationships with the public health and well-being. Aircraft failures during flight are disastrous safety hazards to passengers, crew and people under the flight paths. The suggested predictive maintenance framework will help to increase air travel safety and minimize the risk of aviation accidents caused by the unnoticed mechanical wear, thus contributing to the SDG 3 goal of promoting safe and healthy living among all people.

SDG 17 — Partnerships for the Goals: The project uses NASA C-MAPSS publicly available benchmark dataset, which is a type of open scientific collaboration helping to share

knowledge among the research community. The spirit of SDG 17, which implies the significance of international collaboration and sharing of science, technology, and innovation to achieve sustainable development goals, is further reinforced by the open-source implementation based on PyTorch and XGBoost.

Figure 1.1 United Nations Sustainable Development Goals [23].

1.6.4 Chapter Overview Summary

Chapter 1 — Introduction: It entails the background and motivation, scope and limitations, problem statement, motivation, methodology overview and organization of the thesis where the research context is set and the specific gaps that the proposed hybrid RUL prediction framework is capable of addressing.

Chapter 2 — Literature Review: Provides a critical introduction to the related work in the field of data-driven prognostics and predictive maintenance (Section 2.1), and a survey of existing systems (classical machine learning, deep learning, and hybrid) in the field (Section 2.2). The research gaps identified based on the reviewed literature is formulated in Section 2.3, and the exact objectives of the study based on the research gaps are presented in Section 2.4. Section 2.5 gives a summary of the literature review.

Chapter 3 — Research Methodology: Provides an overview of the research methodology map and how it will be mapped to the project lifecycle (Section 3.1), the tools, technologies, and platforms that will be used (Section 3.2), and the proposed system architecture (Section 3.3). Section 3.4 and 3.5 give a detailed description of data collection and Preprocessing, including

data source and composition, feature selection and structuring, the Preprocessing workflow, sequence construction to perform temporal learning, data splitting and validation strategy, feature engineering, and sequence construction. Section 3.6 gives the end-to-end workflow diagram and Section 3.7 gives a summary of the chapter.

Chapter 4 — Implementation: Records the module-level implementation of the system in three fundamental modules, including data preparation and encoding (Section 4.1.1), sequence construction and windowing (Section 4.1.2), and model architecture and training (Section 4.1.3). Section 4.2 explains the algorithms that the LSTM, XGBoost, and the ensemble parts are based on. The screenshots of the training and the visualizations of its output are introduced in Section 4.3, the integration of the system components is described in Section 4.4, and section 4.5 has the deployment process and an overview of the chapter is introduced in Section 4.6.

Chapter 5 — Results and Discussion: Discloses the experimental setup and the computational environment (Section 5.1), performance metrics to be employed to evaluate the results (Section 5.2), and the results of all three model variants and four sub-datasets (Section 5.3). The results are analyzed and interpreted in detail in Section 5.4, and then the discussion of results and their practical implications is presented in Section 5.5. In Section 5.6, a chapter summary is given.

Chapter 6 — Conclusion and Future Work: Concludes the main contributions of the project (Section 6.1) and gives the key findings (Section 6.2). Section 6.3 and 6.4 discuss the limitations of the study and provide concrete recommendations on the future research directions, such as the quantification of uncertainties, strategies to use in a meta-learner ensemble, validation on real-world data, and the interpretability analysis based on SHAP. Section 6.5 gives a conclusion of the chapters.

Chapter 2

LITERATURE REVIEW

2.1 INTRODUCTION TO RELATED WORK

The practical use of condition-based maintenance in safety-critical industries (such as aerospace, energy and manufacturing) has led to more than 20 years of research on the estimation of Remaining Useful Life (RUL) in industrial machinery, motivated by the practical needs of predictive maintenance. Data-driven prognostics aims to use the data obtained by sensors to create statistical or machine learning systems that can predict the remaining operational life of a component with enough precision and lead time to allow it to be used in maintenance planning [8].

There are three general generations of methods in the research field. The early generation used physics-based degradation models and classical statistics which demanded extensive domain information about failure mechanisms and were not easily extrapolated to new types of machines and operating environments. The second generation presented classical machine learning methods such as Random Forests, Support Vector Regression and gradient boosting methods, which were more flexible and capable of learning predictive patterns using historical run-to-failure data with no explicit physical modelling [1], [3]. Deep learning architectures, specifically recurrent neural networks and their variations dominate the third and current generation and can directly model the time-dependent behavior of sensor degradation sequences without manually designing features [12], [14].

Nevertheless, with this development, there is a structural weakness of the literature: the most popular approaches are still determined by a strong focus on either a temporal sequential structure, by deep learning, or a statistical feature distribution, by classical machine learning, but not on the combination of both in a principled and adapTable way. The proposed hybrid framework is evaluated using the NASA C-MAPSS benchmark which was introduced by Saxena et al. [10] and according to which prognostic approaches are compared and tested across turbofan engine degradation scenarios of different complexity.

In this chapter, the most pertinent published research is critically reviewed in the area of data-driven RUL prediction and predictive maintenance. Section 2.2 provides a review of the

existing systems, laid out along the three generational paradigms above. The section 2.3 specifies the definite research gaps, which inspire the input of this project. These gaps are formal objectives of the study obtained in Section 2.4. Section 2.5 gives an overview of the chapter.

2.2 SURVEY OF EXISTING SYSTEMS

The subsequent survey is based on twelve articles based on conference proceedings and peer-reviewed journals that represent the spectrum of classical machine learning, deep learning, hybrid, and implementation-centered methods of predictive maintenance and RUL estimation. Every work is examined concerning its idea, method, findings, and applicability to the suggested system.

2.2.1 PHM Pipeline Architecture -Jardine et al. [8].

Jardine, Lin and Banjevic created the generic architecture of the Prognostics and Health Management (PHM) systems, surveyed the signal processing, modelling and decision-making roles of an entire condition-based maintenance pipeline [8]. Their key observation is that a successful prognostics solution must not only have good predictive models, but also good sensor signal processing and appropriate integration with maintenance decision processes. The gap marked in this work continues to exist in most of the later literature: the lack of connection between the numerical RUL output of a model and the practical maintenance decision that the model is supposed to be describing. The proposed system fills this gap by a three-tier categorization of maintenance status Critical (RUL $\leq$ 20 cycles) Warning (21-50 cycles) and Healthy (> 50 cycles) that directly converts prediction outputs into maintenance scheduling decisions.

2.2.2 NASA C-MAPSS Benchmark — Saxena et al. [10].

The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) benchmark is a two-spool, high-bypass turbofan engine simulation implemented by Saxena, Goebel, Simon and Eklund and under four sub-configurations of varying complexity of operation: FD001 (one operating condition, one fault mode), FD002 (six operating conditions, one fault mode), FD003 (one operating condition, two fault modes), FD004 (six operating conditions, two fault modes). The benchmark contains multivariate time-series sensor outputs of run-to-failure engine trajectories, and ground-truth RUL values of a held-out test set, and allows

standardized quantitative comparison of prognostic algorithms. C-MAPSS is the standard of the turbofan RUL prediction studies and the only experimental platform in the current study. A critical weakness that the authors note is that the data is provided through simulation, and it might not completely reflect the noise properties and sensor malfunctions that exist in real flight recorder data.

2.2.3 PdM Classical Machine Learning — Patil et al. [1].

A predictive maintenance model in the study by Patil, Jadhav, Bardiya, Davande, and Raverkar utilizes the Random Forest, Support Vector Machine (SVM), and Decision Tree classifiers on sensor data of industrial machines [1]. Their methodology proved to be competitive in classifying faults in tasks where the use of the appropriate feature selection was done, which proved that the classical tree-based method and the use of the kernel methodology can provide meaningful predictive performance in industrial prognostics settings. Nevertheless, the model is based on individual time-step feature observations and fails to capture the time-course properties of sensor readings, being the signals that most effectively capture gradual component degradation. This is the limitation that drives the temporal modelling component of the proposed hybrid framework that directly models the temporal dynamics of sensor degradation using LSTM architecture.

2.2.4 Survey of ML-Based PdM — Sharma et al. [3].

A general survey of machine learning-based predictive maintenance of electrical machines in a large variety of industrial settings was carried out by Sharma, Chauhan, Dwivedi, and Dass [3]. Their main conclusion, namely, the fact that there is no single algorithm, which will work optimally in all machines with all operating conditions, is the pure theoretical incentive of the ensemble approach taken in the given study. The survey equally points to the underrepresentation of multi-condition operating environments in the published literature and absence of systematic approaches to normalization of condition-induced sensor variation. The KMeans-based operating condition normalization and the per-dataset adaptive blending weight α are the main findings of this project and they are directly informed by these observations.

2.2.5 LSTM for RUL Estimation — Zheng et al. [14].

Zheng, Ristovski, Farahat, and Gupta proved that Long Short-Term Memory (LSTM) networks were capable of modelling the turbofan engine degradation in the C-MAPSS benchmark with great accuracy, and they could capture long-range temporal dependencies that are not available to classical feature-based models that work with single-time step observations [14]. Their work made LSTMs the most popular deep learning architecture in time-series RUL prediction and demonstrated that it was possible to learn to do prognostics on raw sensor sequences without heavy manual feature engineering and with strong performance. The LSTM extension of the suggested hybrid model is based on this base and extends the simple recurrent model with a soft attention mechanism and Huber Loss training to overcome the shortcomings of the initial formulation.

2.2.6 Deep Convoluted Networks of RUL — Li et al. [15].

Li, Ding, and Sun were able to show that deep Convolutional Neural Networks (CNNs) were able to generate hierarchical temporal features on sensor sequences and reach accuracy on C-MAPSS with RUL comparable or even better than LSTM networks in some instances [15]. Their effort emphasized the ability of convolutional feature extraction to reach local patterns of degradation at various time scales. Although convolutional models have the benefit of providing computational efficiencies over recurrent models, they do not provide explicit memory mechanisms allowing LSTMs to transfer long-range degradation dynamics over long sequence windows. The current work preserves the LSTM architecture due to its temporal memory capabilities and adds a soft attention mechanism, which offers an efficiently compuTable method of selective weighting of the most informative time steps and does not require the complexity of full self-attention transformers.

2.2.7 Deep Learning Model Comparison — Li and Li [4].

Li and Li have used a systematic comparing of CNN, LSTM and hybrid CNN-LSTM architectures used to predictive maintenance in industrial manufacturing processes [4]. Their findings continuously indicated that hybrid networks performed better than their single type counterparts which give good empirical evidence to the complementary modelling hypothesis that drives the ensemble approach of the current study. Most importantly, though, they were only comparing them to deep learning models and did not consider the possible synergy of deep learning models with other classical gradient boosting models like XGBoost. The

proposed hybrid framework directly fills this gap in the comparison space by integrating an LSTM temporal sequence model with an XGBoost tabular feature model.

2.2.8 CNN-LSTM-Attention on Aero-Engine RUL -Deng and Zhou [17].

A CNN-LSTM-Attention architecture of aero-engine RUL prediction on the C-MAPSS benchmark was proposed by Deng and Zhou, who showed that hybrid CNN-LSTM backbones with an attention mechanism provided better RUL prediction than their counterparts [17]. Their work has offered significant empirical support to the worth of attention mechanisms in the prognostics field, which stimulated the soft attention that is integrated into the LSTM branch of the current framework. The main difference between the proposed system and this work is the clear distinction between the temporal sequence modelling (LSTM branch) and the statistical feature modelling (XGBoost branch) stage coupled with an adaptive learned blending weight, which offers a more interpretable and flexible ensemble structure than a monolithic deep architecture.

2.2.9 Heterogeneous Model Combination Aziz and Sowmyashree [7].

An AI-based predictive maintenance pipeline suggested by Aziz and Sowmyashree was based on an Auto encoder to detect anomalies, Gradient Boosting to predict failures, and an LSTM to estimate the RUL [7]. Their system was able to prove that such heterogeneous combinations of models: having different models do different things in the prognostic pipeline, can be more accurate as a whole than any single model. This article is the thematically nearest to the proposed framework of the reviewed literature. Two very important differences are however: they do not use an optimized, per-dataset blending weight that dynamically decides the relative contribution of each model, and they do not deal with the multi-condition normalization issue of FD002 and FD004. These two gaps are directly handled in the current research.

2.2.10 ML and Deep Learning in Industrial IoT PdM Malaiyappan et al.[2].

Malaiyappan, Krishnamoorthy, and Jangoan reviewed machine learning and deep learning solutions to predictive maintenance in the environment of the Industrial IoT implementation [2]. In their survey, they found that the latency was a key practical limitation in real-time IoT PdM systems, and analyzed different model architectures based on their inference speed and resource consumption. Although the survey is useful to provide the background on the

deployment considerations of PdM systems, it does not address accuracy of RUL prediction or the particular challenges in multi-condition engine operation. The suggested structure applies to this deployment setting in that the LSTM and XGBoost modules generate RUL estimates sufficiently low-level to use with near-real-time scheduling applications.

2.2.11 Deep Learning Ensemble of System RUL - Wang et al. [19].

Wang, Zhu, and Zhao suggested a predictive maintenance approach with dynamic prediction based on a deep learning ensemble model of system-level RUL prediction [19]. Their methodology showed that ensemble techniques that rely on deep learning elements may be employed to provide good generalization in different operating conditions, which supports the main hypothesis of the current study which is the core ensemble hypothesis. The work is referred to directly as the evidence that the performance tendencies identified in the suggested hybrid framework, namely the dominance of ensemble methodology in multi-condition setups, are shared by the literature on deep learning prognostics, in general. Nevertheless, their combination was still in the deep learning paradigm and not the complementarity of tree-based statistical models and recurrent sequence models.

2.2.12 PdM Architecture based on IoT - Singh [6].

A real-time IoT-based predictive maintenance monitoring architecture was proposed by Singh, wherein the system components necessary to complete end-to-end PdM implementation were sensor networks, cloud processing pipelines, and alert generation mechanisms [6]. This literature is applicable to the current study because it places the proposed RUL prediction framework into context in the deployment setting that it is supposed to be used. The maintenance status categorization scheme that was included in the proposed framework - the transformation of numeric RUL predictions into actionable Critical, Warning and Healthy status tags - is directly inspired by the operational alert generation needs outlined in this work. One of the weaknesses of the architecture provided by Singh is the lack of a high-accuracy RUL regression model, which is the exact gap that the proposed hybrid framework addresses.

Table 2.1 Summary of Literature Reviewed

Ref.	Authors	Approach Used	Dataset	Key Finding	Limitation Identified
[1]	Patil et al., 2023	Random Forest, SVM, Decision Trees	Industrial sensor data	Competitive RUL accuracy with feature selection in industrial contexts	Single time-step observation; ignores temporal sensor sequences
[2]	Malaiyappan et al., 2024	ML and Deep Learning survey in Industrial IoT	Industrial IoT	Broad coverage of PdM methods; highlights latency constraints	Focuses on latency rather than RUL prediction accuracy
[3]	Sharma et al., 2025	Survey of ML algorithms for predictive maintenance	Multiple machine types	No single algorithm consistently outperforms across all machine types	Motivates ensemble approach; no combined model proposed
[4]	Li and Li, 2025	CNN, LSTM, CNN-LSTM comparison	Manufacturing process data	Hybrid CNN-LSTM consistently outperforms single-type	No XGBoost ensemble or attention mechanism considered

Ref.	Authors	Approach Used	Dataset	Key Finding	Limitation Identified
				networks	
[5]	Garcia et al., 2025	NLP-assisted review of condition monitoring and PdM	Literature corpus	Highlights poor coverage of deployment feasibility in published work	Review study only; no experimental contribution
[6]	Singh, 2025	IoT-based real-time monitoring architecture for PdM	Industrial machinery	Demonstrates feasibility of real-time IoT PdM deployment	No RUL regression model; focuses on architecture not accuracy
[7]	Aziz and Sowmyashree, 2025	Auto encoder + Gradient Boosting + LSTM pipeline	Industrial equipment	Heterogeneous model combination improves pipeline accuracy	No adaptive blending weight; no multi-condition normalization
[8]	Jardine et al., 2006	PHM pipeline review: signal processing,	General machinery	Foundational PHM architecture; identifies gaps in maintenance	Maintenance decision integration still under-researched in

Ref.	Authors	Approach Used	Dataset	Key Finding	Limitation Identified
		modelling, decision-making		decision integration	later literature
[9]	Abouloifa and Bahaj, 2025	Deep learning vs. Fourier series for failure prediction	Equipment sensor data	Rigorous statistical comparison of deep vs. classical approaches	Does not address RUL estimation or multi-condition environments
[10]	Saxena et al., 2008	C-MAPSS simulation benchmark for turbofan degradation	NASA C-MAPSS (FD001–FD004)	Establishes the standard benchmark for systematic prognostic comparison	Simulated data; real-world noise characteristics may differ
[14]	Zheng et al., 2017	LSTM network for RUL estimation	NASA C-MAPSS	LSTMs model long-range temporal dependencies superior to classical methods	No attention mechanism; no ensemble with tree-based models
[17]	Deng and	CNN-LSTM-	NASA C-	Attention mechanism	No XGBoost ensemble

Ref.	Authors	Approach Used	Dataset	Key Finding	Limitation Identified
	Zhou, 2024	Attention for aero-engine RUL	MAPSS	improves RUL accuracy on C-MAPSS benchmark	branch; no multi-condition normalization strategy

2.3 RESEARCH GAPS IDENTIFIED

The above literature review of the current systems shows that there are four distinct research gaps that when combined can be seen as driving the proposed hybrid framework. These gaps are expressed as follows by their importance to the design choices of the current study.

Gap 1 - Lack of Principled Cross-Paradigm Ensemble Strategies: Although multiple surveys [3] and comparative studies [4] have shown that no single modelling paradigm is uniformly dominant across operating situations, the literature does not describe any published work that integrates an attention-augmented LSTM with an XGBoost regressor using a principled adaptive combination of blending weights optimized on a Other works like Aziz and Sowmyashree [7] show the usefulness of heterogeneous model combination but fail to provide a data-driven process of the determination of the optimal relative contribution of each factor. This is the main gap that is served by the proposed framework.

Gap 2 - Poor Multi-Condition Sensor Normalization: The C-MAPSS sub-datasets FD002 and FD004 model 6 different operating conditions, which induce systematic sensor baseline changes, which can be attributed to variation in flight regime, but not component degradation. Most of the published methods use global normalization without considering this confounding structure [3], [5], and end up with models that partially mix condition-induced sensor variation with actual degradation signals. None of the published literature on C-MAPSS suggested the use of per-cluster Z-score normalization based on clusters of operating conditions identified by KMeans as a Preprocessing step to both deep-learning and gradient-boosting branches.

Gap 3 — Under-Exploitation of Attention Mechanisms in Turbofan Prognostics: Soft attention mechanisms have been shown to be transformative in natural language processing [13] and have been used successfully in some small number of deep learning prognostic models [17], [18]. But their integration in a hybrid ensemble model that incorporates a recurrent sequence model and a tree-based tabular model has not been investigated. Moreover, most attention-augmented prognostic models in the literature use convolutional-recurrent hybrids as opposed to pure LSTM architectures, and it is hard to distinguish the role of the attention mechanism against the convolutional feature extraction component.

Gap 4 — Lack of Closure between Model Results and Practical Maintenance Actions: It is true to say, as has been found in the literature review on Jardine et al. [8], as has been more recently observed by Garcia et al. [5] and Singh [6], that most published RUL prediction systems currently report accuracy metrics - RMSE, MAE, R^2 - without converting model outputs into the risk An operational useful numerical RUL estimate has to be given in terms of a risk categorization that is interpretable and this translation step is mostly missing in the prognostics literature.

2.4 OBJECTIVES OF THE STUDY

Based on the four research gaps identified in the previous section, the objectives below determine the scope and contributions that the current research is expected to make:

Objective 1 — To design and apply a hybrid ensemble framework, which is an attention-augmented LSTM network and an XGBoost regressor, using a per-dataset learned blending weight $\boldsymbol{\alpha}$, optimized by grid search with a held-out validation partition, to leverage the complementary capabilities of time sequence modelling and feature modelling by statistical methods to predict RUL on the NASA C-MAPSS benchmark.

Objective 2 — To design and test a KMeans-based operating condition normalization strategy which learns the main flight regimes in the multi-condition sub-data (FD002 and FD004) using unsupervised clustering, and uses per-cluster Z-score normalization to decouple sensor baseline shifts and true degradation indicators, thus allowing both branches of the model to learn degradation dynamics, not

Objective 3 — To introduce a soft attention mechanism into the LSTM branch of the hybrid model, such that the model will learn to weight the most informative time steps in the 50-

cycle sliding window sequence of model inputs, and evaluate the effect of this mechanism on model convergence and prediction accuracy compared to an attention-free LSTM baseline.

Objective 4 — To convert the numerical RUL predictions of the hybrid ensemble into a three-tier maintenance status category system Critical (RUL ≤ 20 cycles), Warning (21–50 cycles**)**, and Healthy (> 50 cycles) to close the model output to operational deployment context maintenance scheduling decision gap.

Objective 5 — To compare the performance of all three variants of the model standalone LSTM with Attention, standalone XGBoost, and Hybrid Ensemble to all four NASA C-MAPSS sub-datasets on the metrics of RMSE, MAE, and R^2 and report the findings without

2.5 SUMMARY

This chapter provided a critical analysis of the available literature on the data-driven Remaining Useful Life prediction and predictive maintenance, based on twelve published papers in the form of conference papers and peer-reviewed journals. The survey followed the history of the development of the prognostic strategies starting with the fundamental PHM pipeline architecture of Jardine et al. [8] and the creation of the NASA C-MAPSS benchmark by Saxena et al. [10], to the classical machine learning methods of Patil et al. [1], the complete surveys of Sharma et al. [3] and Garcia et al. [5], up to the deep learning

The survey identified four distinct research gaps in the current literature: the lack of principled cross-paradigm ensemble strategies integrating LSTM and XGBoost using an adaptive learned blending weight; the limited coverage of multi-condition sensor normalization on multi-flight regime datasets; the under coverage of soft attention mechanisms in hybrid ensemble models; and the continued disconnection between the outputs of numerical RUL and the actionable maintenance scheduling decision making

Out of the above gaps, five formal study objectives were formulated, including hybrid ensemble design and implementation, KMeans-based normalization, attention mechanism integration, maintenance status categorization, and rigorous comparative analysis of all model variants and sub-datasets. The suggested framework directly focuses on all the four gaps identified and is set to meet all the five objectives stated. Chapter 3 explains in detail the methodology by which these objectives are to be approached.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 RESEARCH METHODOLOGY OUTLINE

The research methodology adopted in this project follows a structured, iterative data science lifecycle tailored to the requirements of machine learning-based prognostics research. The methodology is designed to ensure that all modeling decisions are traceable to clearly defined requirements, that the evaluation process is rigorous and reproducible, and that the resulting framework is both technically sound and practically relevant to real-world turbofan engine maintenance contexts.

The methodology is organized into six sequential phases. The first phase, Problem Definition and Literature Review, involved a systematic survey of the existing prognostics literature to identify specific research gaps and formulate the formal objectives of the study, as documented in Chapter 2. The second phase, Data Acquisition and Preprocessing, involved obtaining the NASA C-MAPSS benchmark dataset, implementing the sensor filtering pipeline, designing the operating condition normalization strategy, constructing the RUL label capping scheme, and building the sliding window sequence generator, as detailed in Sections 3.4 and 3.5.

The third phase, Feature Engineering, involved the construction of the 73-dimensional feature vector comprising raw operating condition settings, filtered sensor values, and four rolling window statistical features per sensor, as described in Section 3.5.2. The fourth phase, Model Design and Implementation, involved the architectural specification and implementation of the two modeling branches — the attention-augmented LSTM and the XGBoost regressor — along with the hybrid ensemble blending mechanism, as described in Section 3.3 and Chapter 4. The fifth phase, Training and Optimization, involved executing training runs for all model variants across all four sub-datasets, including the grid search Optimization of the per-dataset blending weight α. The sixth phase, Evaluation and Analysis, involved computing RMSE, MAE, and R^2 for all model variants and interpreting the results, as documented in Chapter 5.

3.2 TOOLS, TECHNOLOGIES, PLATFORMS USED

The proposed hybrid RUL prediction framework was implemented using a carefully selected stack of open-source tools and libraries, chosen for their suitability to the computational requirements of the project and their broad adoption in the machine learning research community. Table 3.1 provides a comprehensive summary of all tools, technologies, and platforms used in the project, along with their respective versions and roles within the system.

Table 3.1 Tools, Technologies, and Platforms Used

Category	Tool / Library	Version	Purpose in Project
Programming Language	Python	3.9	Core development language for all pipeline modules
Deep Learning	PyTorch	2.x	LSTM model construction, training, and inference
Gradient Boosting	XGBoost	1.7	Tabular regression branch training and prediction
Data Processing	NumPy, Pandas	Latest sTable	Dataset loading, feature engineering, rolling statistics
ML Utilities	Scikit-learn	Latest sTable	KMeans clustering, MinMaxScaler, data splitting
Visualization	Matplotlib, Seaborn	Latest sTable	All result plots, training curves, scatter plots
Development Environment	Google Colab / Jupyter	—	Interactive experimentation and notebook execution

Hardware	Tesla T4 GPU	15.6 GB VRAM	GPU-accelerated LSTM training
Dataset	NASA C-MAPSS	Public benchmark	Turbofan engine run-to-failure simulation data
Deployment	Streamlit	Latest sTable	Interactive web app for RUL prediction and maintenance status display
Version Control/Hosting	Git / GitHub	—	Source code management, model artefact storage, and CI/CD trigger for Streamlit deployment

Python 3.9 was selected as the core development language due to its extensive ecosystem of machine learning, data processing, and scientific computing libraries. PyTorch 2.x provided the deep learning framework for constructing, training, and evaluating the LSTM with Attention branch. XGBoost 1.7 was used for the tabular gradient boosting branch, providing efficient tree ensemble training with built-in regularization and early stopping. NumPy and Pandas handled all dataset loading, Preprocessing, and feature engineering operations. Scikit-learn provided the KMeans clustering implementation, MinMaxScaler, and data splitting utilities. Matplotlib and Seaborn were used for all result visualizations. All training experiments were executed on a Tesla T4 GPU with 15.6 GB of VRAM accessed through a cloud-based notebook environment.

3.3 PROPOSED ARCHITECTURE

Our hybrid RUL prediction model takes a dual path parallel structure so that the same input data is processed by two parallel and complementary modeling paradigms, followed by a combination of their results using an adaptive weighted ensemble. This design is driven by the first research gap specified in Chapter 2: the lack of any single modeling paradigm that can always take advantage of both the time dynamics and the statistical features distributions within the sensor data.

The overall end-to-end system architecture is illustrated in Figure 3.1. Raw C-MAPSS sensor data is processed into the pipeline and is preprocessed through sensor filtering, normalization of operating conditions, capping of RUL labels at 125 cycles and the 73-dimensional feature vectors are built. The processed data is then inputted to two parallel branches in a preprocessed form.

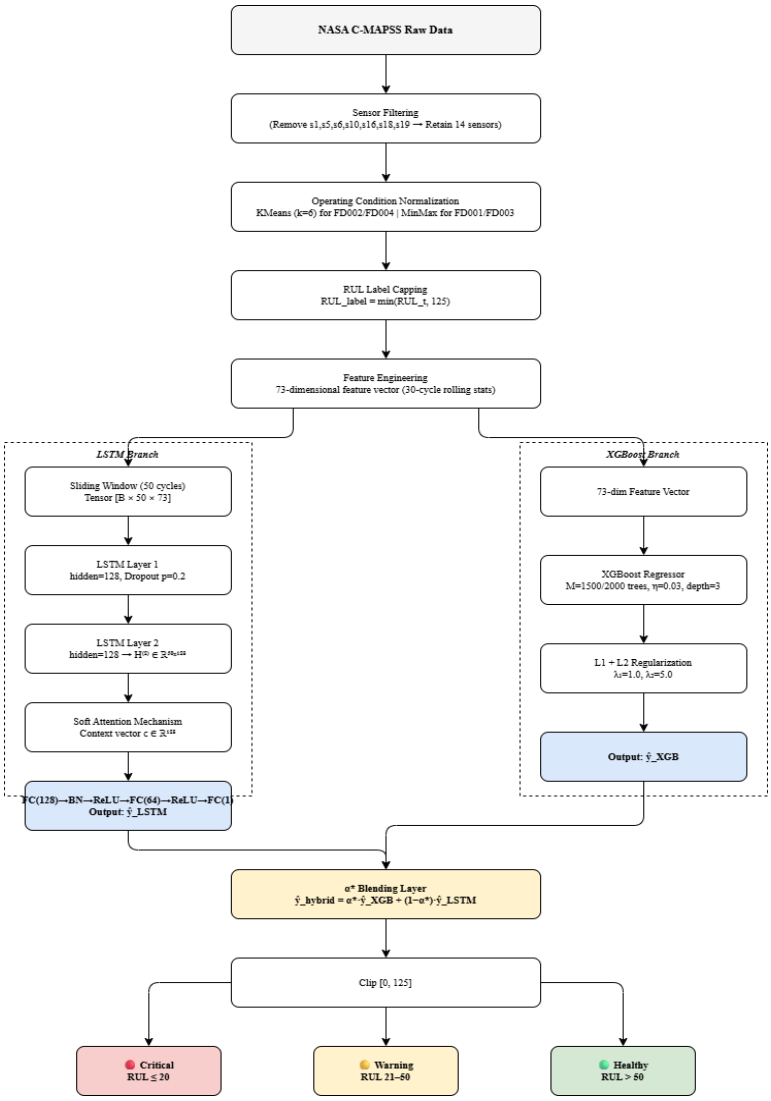


Figure 3.1 End-to-end system architecture.

The original branch, the Sequential Temporal Branch, takes the data in the form of ordered time-series sequences of size [Batch × 50 × 73] with 50 being the length of the sliding window in cycles and 73 the number of features. This branch uses two stacked LSTMs with a soft attention mechanism, and a fully connected regression head. The LSTM model is an architecture that learns the temporal dynamics of engine degradation as it undergoes its

operating history, and the attention mechanism allows the model to focus on the most informative engine degradation-phase cycles in predicting the RUL [12], [13]. A LSTM with Attention branch is illustrated in Figure 3.2.

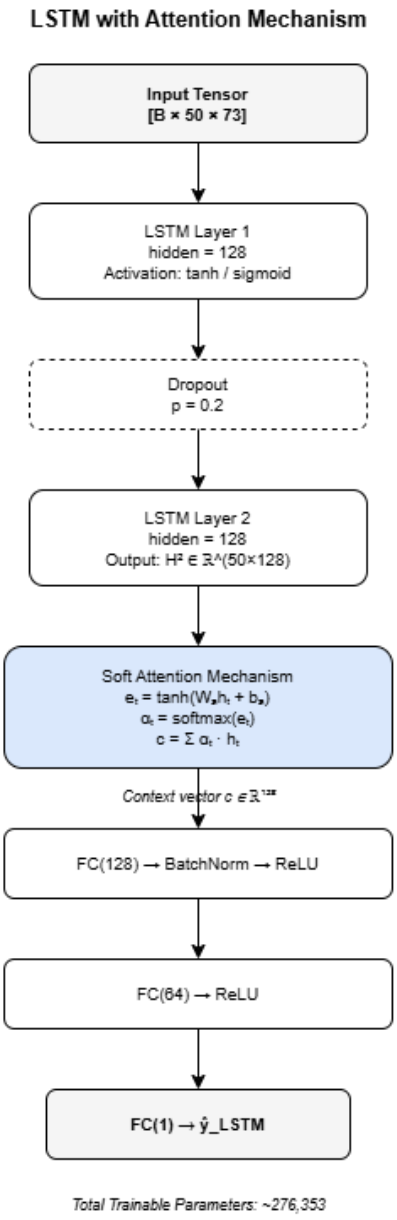


Figure 3.2 LSTM with Attention architecture.

The second branch is the Statistical Feature Branch which processes the identical data as a flat 73-dimensional feature vector, but does not order the snapshots of features across the cycles of a cycle. In this branch, an XGBoost gradient boosted tree ensemble [11] is used, which is also effective in non-linear statistical relationships and feature interactions in the engineered feature space. Figure 3.3 represents the XGBoost architecture.

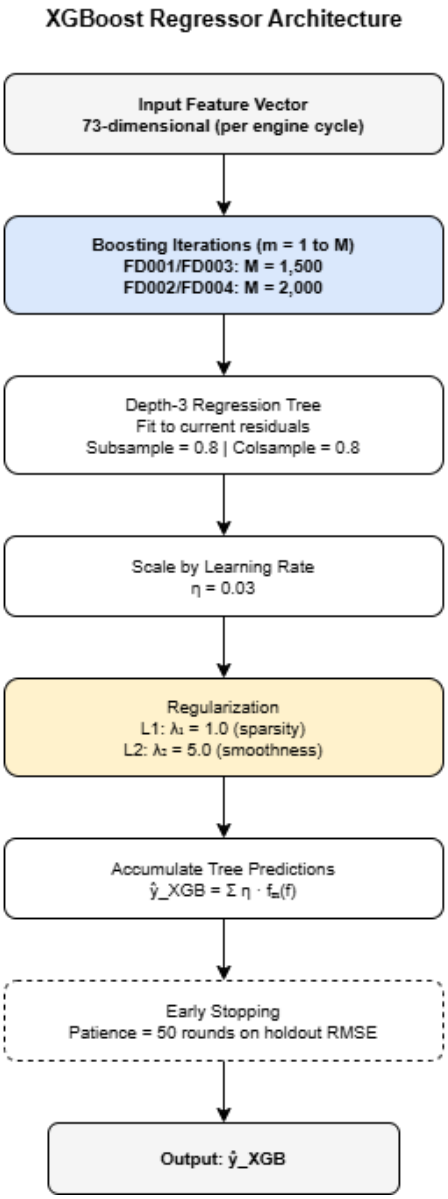


Figure 3.3 XGBoost architecture.

The results of the two branches are ŷ_LSTM and ŷ_XGB, which are then fused in a linear fusion: ŷ_hybrid = α · ŷ_XGB + (1 − α) · ŷ_LSTM. Optimization of the blending weight α is done on a held-out validation partition by grid search in [0, 1] with a step size of 0.01. The last prediction range is trimmed to [0, 125] and converted into a category of maintenance status: Critical (RUL ≤ 20), Warning **(21–50)**, or Healthy (> 50). Figure 3.4 represents the hybrid ensemble blending mechanism.

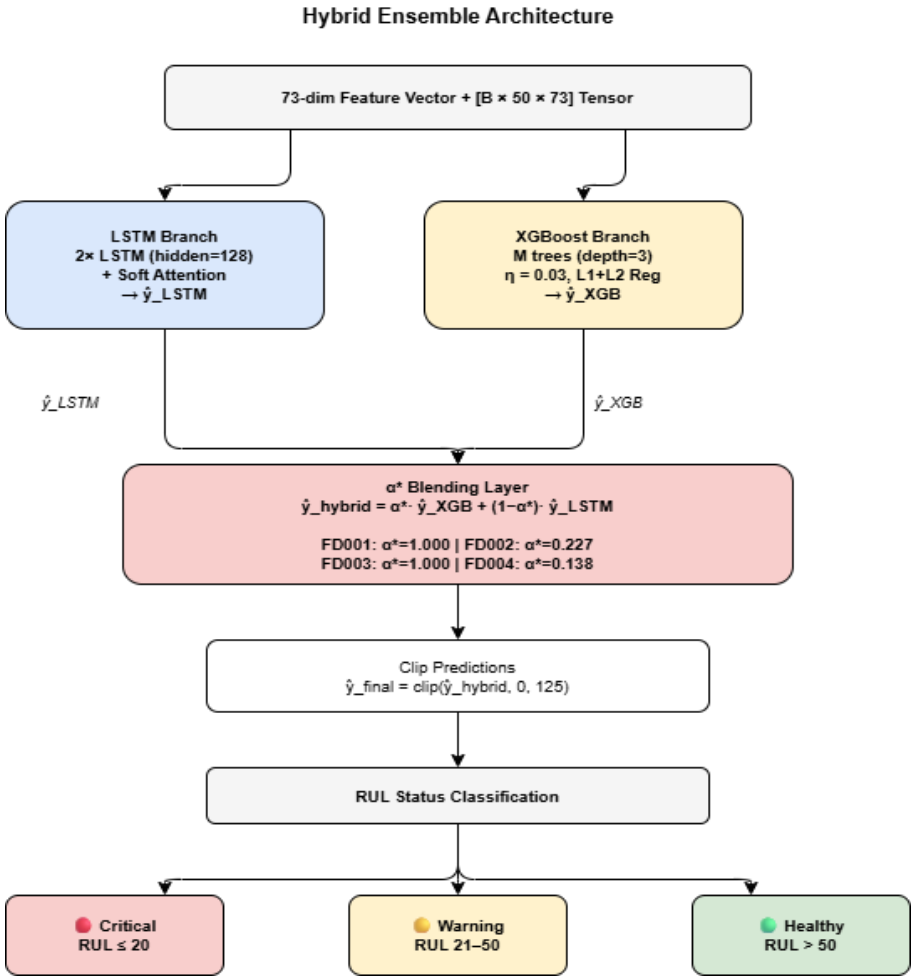


Figure 3.4 Hybrid ensemble blending mechanism

3.4 DATA COLLECTION AND PREPROCESSING

The initial stage in the proposed methodology is the data collection and Preprocessing phase. The quality and suitability of the Preprocessing pipeline directly affect both the quality of features offered to each of the two model branches, and is especially important in multi-condition nature of the FD002 and FD004 sub-datasets. Figure 3.5 shows the entire Preprocessing pipeline.

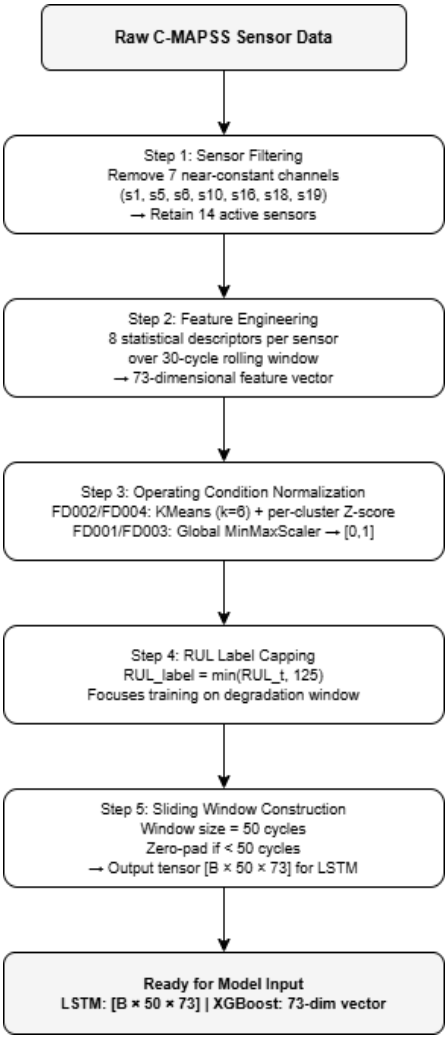


Figure 3.5 Preprocessing pipeline

3.4.1 Data Source and Composition

This section discusses the data source and its composition. The information in this project comes out of NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) data available online through NASA Prognostics Data Repository [10]. The C-MAPSS simulator has a two-spool high-bypass turbofan engine, instrumented with 21 sensors and three operating condition conditions and produces multivariate time-series data over run-to-failure engine runs. The data is divided into four sub-data sets, namely FD001, FD002, FD003 and FD004, with different number of operating conditions and fault modes simulated in each, as summarized in Table 3.2.

Table 3.2 NASA C-MAPSS Sub-dataset Configurations

Sub-dataset	Operating Conditions	Fault Modes	Train Sequences	Test Engines	Normalization
FD001	1	1	15,631	100	MinMax
FD002	6	1	40,759	259	KMeans Z-score
FD003	1	2	19,720	100	MinMax
FD004	6	2	48,799	248	KMeans Z-score

The data is represented as records of the following form: unit identifier, cycle number, three operating condition settings (altitude, Mach number, throttle resolver angle), and 21 sensor measurements. The training files include full run-to-failure trajectories of all training engines, and the test files include truncated trajectories cut off at a censored time point, and an independent ground-truth file of the actual remaining life at truncation, called RUL. The large difference in the number of training sequences between FD001 (15,631) and FD004 (48,799) is due to the higher quantity of engines and longer working paths of the multi-condition sub-datasets.

3.4.2 Feature Selection and Structuring

The raw data is a set of 21 sensor channels, not all of which will have useful evidence on degradation. The training data underwent a variance-based filtering analysis to determine and eliminate sensor channels that have values that are effectively constant throughout the engine dynamics and operating conditions. The researchers identified seven sensor channels (s1, s5, s6, s10, s16, s18, and s19) as having a near-zero variance and dropped them out of the feature set, since they do not offer any discriminative information in estimating RUL. Table 3.3 lists the 14 retained sensor channels and their physical interpretation and degradation trends.

Table 3.3 Active Sensor Channels after Variance-Based Filtering

ID	Measurement	Trend	Physical Role	Status
s2	Fan Inlet Temperature	Increasing ↑	Thermal load indicator	Retained
s3	LPC Outlet Temperature	Increasing ↑	Compressor stress	Retained
s4	HPC Outlet Temperature	Increasing ↑	Primary degradation marker	Retained
s7	HPC Outlet Pressure	Decreasing ↓	Compression efficiency	Retained
s8	Fuel Flow to Burner	Increasing ↑	Fuel efficiency loss	Retained
s9	Bypass Ratio	Decreasing ↓	Engine performance	Retained
s11	HPC Outlet Temperature (Redundant)	Increasing ↑	Redundant thermal check	Retained
s12	Fan / Core Speed Ratio	Decreasing ↓	Mechanical degradation	Retained
s13	Corrected Fan Speed	Decreasing ↓	Fan wear indicator	Retained

ID	Measurement	Trend	Physical Role	Status
s14	Corrected Core Speed	Increasing ↑	Core stress	Retained
s15	Bypass Ratio (Secondary)	Decreasing ↓	Secondary performance check	Retained
s17	Bleed Enthalpy	Decreasing ↓	Bleed valve condition	Retained
s20	HPT Cool Air Flow	Decreasing ↓	Cooling efficiency	Retained
s21	LPT Cool Air Flow	Decreasing ↓	Turbine health	Retained
s1,s5,s6,s10,s16,s18,s19	Various (near-constant channels)	No trend	No degradation information	Removed

The last structured feature set of each engine cycle consists of three operating condition settings, 14 retained sensor values and 56 engineered rolling window features as presented in Section 3.5.2, giving a total of 3 + 14 + 56 = 73 features per cycle. This feature of 73 dimensions is the input of both branches of the model.

3.4.3 Preprocessing Workflow

After sensor filtering and feature structuring, Preprocessing workflow passes through three consecutive transformations namely operating condition normalization, RUL label construction and capping, and sliding window sequence construction. Normalization of Operating Conditions: Differentiation on the number of operating conditions in each sub-dataset is applied to the normalization strategy.

In the single-condition sub-datasets FD001 and FD003, a global MinMaxScaler is trained on the training data of each of the 14 sensor channels and three operating condition settings, and all values are mapped to the range [0, 1]. This method is suitable since all engine cycles in these sub-datasets belong to one operating regime and thus a global scalar is right to capture the entire range of sensor variation that can be due to degradation. I

n the case of multi-condition sub-datasets FD002 and FD004, a KMeans model with k = 6 is trained on the three operating condition columns of the training data. Per-cluster means and standard deviations are then calculated with respect to the sensor channel and per-cluster Z-score normalization is done based on the resulting cluster assignments.

This plan separates sensor baseline shift caused by changes in the flight regime with the actual degradation signal, allowing both model branches to learn actual degradation dynamics. RUL Label Construction and Capping: Ground-truth RUL labels are built at each training cycle with RUL(t) = T_failure − t where T failure is the sum of cycles run on the engine. Then a piecewise cap is used: RUL_label = min(RUL(t), 125).

This limit so that the model does not waste representational capacity in the early healthy portion of the engine lifecycle where sensor data gives the most insignificant prognostic clues. The capping effect on a representative FD001 engine trajectory is illustrated in Figure 3.6.

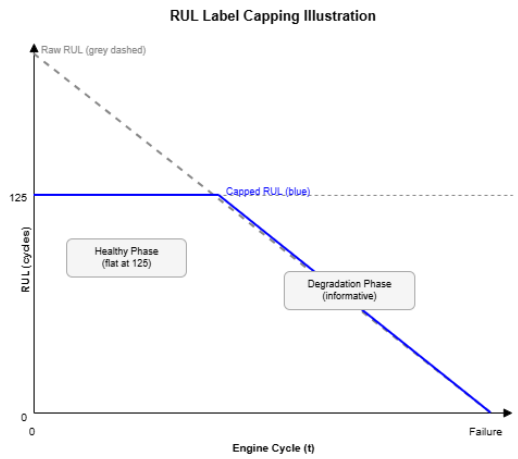


Figure 3.6 RUL label capping for a representative FD001 engine.

3.4.4 Sequence Construction for Temporal Learning

The LSTM branch will need its input to be presented in the form of ordered temporal sequences as opposed to snapshots of individual cycles. The sliding window algorithm is

used and the window size is fixed to SEQ_LEN = 50 cycles. Each engine and each cycle t are assigned a sequence consisting of the 73-dimensional feature vectors of the 50 most recent cycles until and including cycle t. Engines with a cumulative lifetime of less than 50 cycles are processed with zero-padding, whereby the feature vectors are prefixed with zero values to make the sequence length 50.

The input tensors to the LSTM branch are the resulting tensors and are of shape [B × 50 × 73], with B being the batch size. In the XGBoost branch, the 73-dimensional feature vector of cycle t is taken as a flat input, without time order. This sliding window construction is such that each labeled cycle in the training set produces a specific training sample, so that as much run-to-failure data as possible is utilized.

3.4.5 Data Splitting and Validation Strategy

In order to conduct a rigorous and impartial analysis, data is divided at the engine level as opposed to cycle level. Cycle-level splitting would introduce the risk of leakage in time-data, with the cycles of the same engine being present in the training and validation sets, and the model would then use engine-specific degradation information instead of learning generalisable degradation behaviour. Engine level splitting guarantees that every cycle of a particular engine is allocated to either the training, validation or the hold out partition.

The ratios used in the split are 85 percent training, 10 percent validation, and 5 percent holdout. LSTM early stopping and grid search optimization of α are done on the validation partition. Holdout partition Holdout partition is only accessed during the last performance evaluation reported in Chapter 5 and is never used during training or hyperparameter selection. The resulting partition sizes of each sub-dataset are summarized in Table 3.4

Table 3.4 Engine-Level Data Split Across Sub-datasets

Sub-dataset	Train (85%)	Validation (10%)	Holdout (5%)	Split Level
FD001	13,287	1,563	781	Engine-level
FD002	34,647	4,075	2,037	Engine-level

Sub-dataset	Train (85%)	Validation (10%)	Holdout (5%)	Split Level
FD003	16,762	1,972	986	Engine-level
FD004	41,481	4,879	2,439	Engine-level

3.5 DATASET DESCRIPTION

3.5.1 Data Acquisition and Understanding

The NASA C-MAPSS data has become the most popular benchmark on the study of turbofan engine RUL prediction research and was also released together with the PHM 2008 Data Challenge by Saxena et al. [10]. The dataset was directly downloaded through the NASA Prognostics Data Repository and read into the experimental pipeline using Pandas to process the data in tabular form. The standard column schema of unit id, cycle, op1, op2, op3, s1 to s21 was loaded in each of the four sub-dataset training files.

The ground-truth files in the corresponding RUL are the true remaining life of each test engine at a trajectory truncation point. The preliminary exploratory analysis was used to validate the reported features of each sub-dataset. The sensor trends of FD001 and FD003 are relatively clean and monotonically degrading and therefore can be easily modeled sequentially. FD002 and FD004 share the typical multi-modal sensor distributions due to the six different flight regimes that will present as sharp changes in the baseline sensor readings that are entirely due to changes in the operating conditions and not component degradation.

This exploratory analysis directly confirmed the design choice to use KMeans-based per-cluster normalization on FD002 and FD004 only. After Preprocessing and building of the sliding window, 124,909 training sequences were obtained in all four sub-datasets.

3.5.2 Feature Engineering and Sequence Construction

The feature engineering is used to add temporal statistical features to the raw sensor data that represent various properties of the degradation dynamics in a rolling 10-cycle window. On each sensor channel j and on each cycle t, four statistical features are calculated over the

window of the 10 most recent cycles. The Rolling Mean takes the absolute position of the sensor, which gives a smoothed indication of the current running condition. Rolling Standard Deviation is used to capture the short-term variability that is likely to rise as the components start to deteriorate.

The Linear Slope is an encoding of local direction and magnitude of trends in the sensor, but the rate of change of the sensor across the window. Exponentially Weighted Moving Average (EWMA) with the decay factor $\alpha = 0.1$simply gives precedence to the latest cycles in the window, and gives a Recency-weighted forecast of the sensor path. These four features are described in Table 3.5.

Table 3.5 Engineered Features per Sensor — 10-Cycle Rolling Window

Feature Name	Window	Mathematical Basis	Aspect Captured
Rolling Mean (μ)	10 cycles	Average of sensor values over last 10 cycles	Absolute sensor level
Rolling Std (σ)	10 cycles	Standard deviation over last 10 cycles	Short-term variability
Linear Slope (β)	10 cycles	Least-squares linear trend over last 10 cycles	Rate of degradation
EWMA (α=0.1)	10 cycles	Exponentially weighted moving average, recent cycles weighted more	Recency-weighted trend

Using the four features on each of the 14 retained sensor channels would result in 14 x 4 = 56 engineered features. Together with 14 raw sensor values and 3 operating condition settings, the resulting feature vector dimension is 3 + 14 + 56 = 73. The 73-dimensional representation is the input to both branches of the model: as a time-ordered sequence of 50 consecutive vectors to the LSTM branch, and as a flat single-dimensional vector to the XGBoost branch.

3.5.3 Synthetic Data Generation Using Rolling Window Augmentation

The sliding window construction scheme that has been used in this project is an implicit data augmentation that makes the training samples very large in comparison with the number of distinct engine run-to-failure trajectories in the dataset. To every engine having N overall operating cycles, the sliding window method produces N training sequences labeled (one per cycle, including zero-padding). The augmentation is especially useful in sub-datasets that have moderate sized training engines, but long operational trajectories, in which the sliding window increases the effective size of the training set by a factor that is a proportion of the average length of the operational trajectory.

In the case of FD001, the mean trajectory length of engines is about 206 cycles, or each of the training engines adds about 206 labeled sequences. On the 85% training part of FD001 this gives about 13,287 training sequences. The augmentation effect is strongest in FD004 where multi-condition operation gives longer trajectories and more training engines, giving 41,481 training sequences with the 85% training partition alone. This sliding window augmentation approach adds no artificial or synthetic generated data; all feature values are obtained directly as an outcome of observed sensor measurements and are in full accordance with the general practice in time-series deep learning prognostics [14], [17].

3.6 WORKFLOW DIAGRAM

Figure 3.7 shows the entire end-to-end process of the proposed hybrid RUL prediction framework. The figure shows the chronological sequence of data and decisions in the raw C-MAPSS input to Preprocessing and feature engineering, parallel model training, ensemble Optimization, and final evaluation. Every section of this chapter or a module as explained in Chapter 4 is associated with a given stage of the workflow.

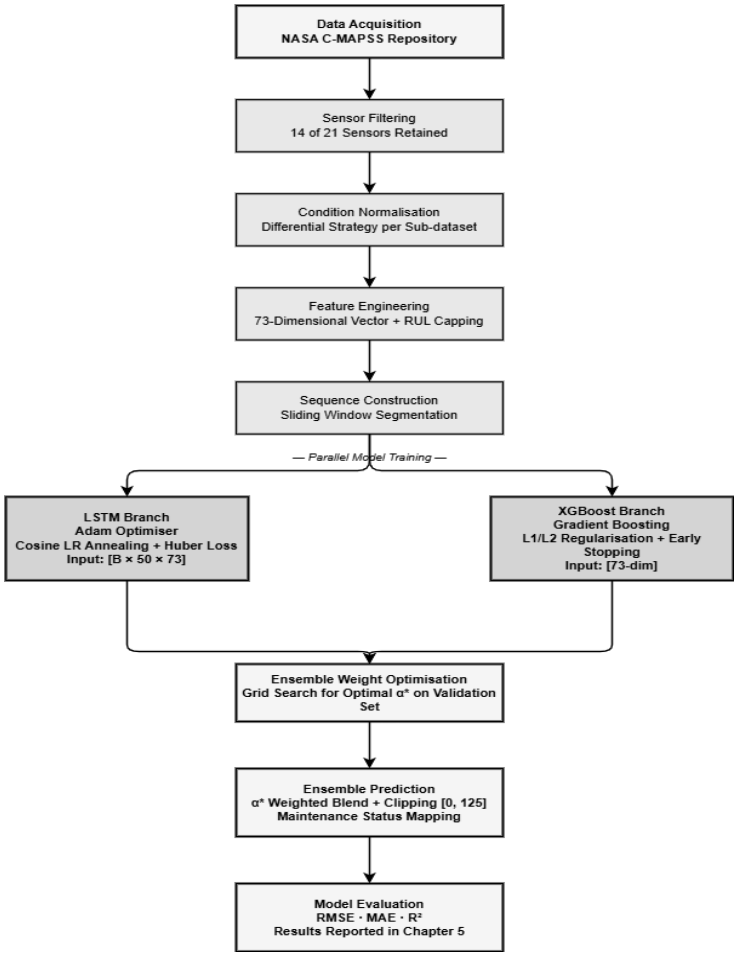


Figure 3.7 End-to-end workflow of the proposed hybrid LSTM-XGBoost framework.

It starts with the acquisition of data in NASA C-MAPSS repository and continues with sensor filtering, normalization of operating conditions (differential strategy depending on sub-dataset complexity), label capping of RUL, and construction of a 73-dimensional feature vectors. The processed data is collected into sliding window sequences of the LSTM branch and flat tabular inputs of the XGBoost branch. Both branches are trained separately: the LSTM is trained with Adam optimizer with Cosine learning rate annealing and Huber Loss, XGBoost is trained with gradient boosting with L1/L2 regularization and early stopping.

Once it has been trained, the blending weight α* is calculated by using grid search on the validation partition. The hybrid prediction is achieved by combining both the branch outputs with α*, and the clipping to [0,125], and mapping to maintenance status categories. The held-out test set is evaluated using RMSE, MAE, and R^2, and the full results are provided in Chapter 5.

3.7 SUMMARY

In this chapter, the entire research methodology on which the proposed hybrid LSTM-XGBoost ensemble framework, the turbofan engine RUL prediction, is based was outlined. The methodology was introduced in the form of six-phase lifecycle of data science that involves problem definition, data acquisition and Preprocessing, feature engineering, model design and implementation, training and Optimization, and evaluation and analysis. Table 3.1 summarized the tools and technologies used in the project, including Python-based development stack, such as PyTorch 2.x, XGBoost 1.7, Scikit-learn, Tesla T4 hardware platform, and Streamlit and GitHub for deployment.

Section 3.3 gave a description of the proposed two-branch parallel architecture, where the attention-augmented LSTM did the temporal sequence modeling and the XGBoost did the statistical feature modeling, which were integrated via an adaptive weighted ensemble. Section 3.4 outlined the data collection and Preprocessing pipeline in five subsections that dealt with the NASA C-MAPSS data source and composition (Table 3.2), sensor filtering and feature selection (Table 3.3), the differential normalization strategy, the RUL capping strategy, sequence construction and the engine-level data splitting strategy (Table 3.4).

Section 3.5 has recorded data acquisition insights and the 73-dimensional feature engineering scheme (Table 3.5) and the sliding window augmentation strategy. The workflow diagram of Section 3.6 summarized the end-to-end workflow. All modules mentioned in this chapter are implemented in Chapter 4 in a detailed way.

CHAPTER 4

IMPLEMENTATION

This chapter records all processes of the proposed hybrid LSTM-XGBoost ensemble structure in predicting turbofan engine RUL. The implementation is structured into three main modules each of which is linked to a specific phase of the pipeline as outlined in Chapter 3. Section 4.2 gives the formal descriptions of the important computational components in plain prose. Section 4.3 provides the pictorials of the training console outputs generated in the experimental runs as the evidence of successful implementation. Section 4.4 explains how all the parts are incorporated into a complete end-to-end system. Table 4.1 gives a summary of the three implementation modules and their duties.

Table 4.1 Implementation Module Overview

Module	Name	Key Responsibilities	Output Produced
M1	Data Preparation and Encoding	Sensor filtering, KMeans clustering, MinMax/Z-score normalization, RUL label capping, feature engineering	Normalized 73-dim feature vectors with capped RUL labels for all engines
M2	Sequence Construction and Windowing	Sliding window assembly (SEQ_LEN=50), zero-padding, LSTM tensor construction, XGBoost flat vector extraction	LSTM input tensors [B×50×73] and XGBoost input matrix [B×73]
M3	Model Architecture and Training	LSTM+Attention model definition, XGBoost fitting, Huber Loss training loop, cosine LR annealing, alpha grid search Optimization	Trained LSTM weights, XGBoost model, optimal α* per sub-dataset

4.1 MODULE-WISE IMPLEMENTATION DETAILS

The implementation is divided into three logically separated modules; the Data Preparation and Encoding Module (M1), the Sequence Construction and Windowing Module (M2), and the Model Architecture and Model Training Module (M3). The modules are tesTable and have clearly defined outputs that are used as inputs by the next module hence modularity, reproducibility, and easy debugging.

4.1.1 Data Preparation and Encoding Module

The Data Preparation and Encoding Module performs the task of loading the raw C-MAPSS dataset, sensor filtering, operating condition normalization, RUL label construction, and generation of the structured 73-dimensional feature vectors which are input to Module M2. This module summarizes all the data-dependent logic and is run separately applying dataset-specific parameterization of configuration parameters to each of the four C-MAPSS sub-datasets. It starts by loading the raw training and test files in structured Data Frames using the standard C-MAPSS column schema.

The seven near-constant sensor channels found during variance analysis (s1, s5, s6, s10, s16, s18, s19) are eliminated in training and test Data Frame. Single-condition sub-datasets FD001 and FD003. In the case of single-condition sub-datasets FD001 and FD003 a MinMaxScaler is trained on the training sensor data and the resulting learned parameters are applied to test and training data to avoid any data leakage. In case of multi-condition sub-datasets FD002 and FD004, a KMeans model with k = 6 is trained on the three columns of operating conditions of the training data. Per-cluster means and standard deviations of each sensor channel, and per-cluster Z-score normalization are computed using the resulting cluster assignments.

The cluster centroid and normalization parameters that have been fitted on the training set are stored and reused when normalizing test sets. Each training cycle is built using RUL (t) = T_failure − t and the piecewise limit of 125 cycles is used. Four rolling window statistical indicators are then calculated on each of the 14 retained sensor channels in 10 cycles, producing 56 further engineered columns. Each sub-dataset produces the final encoded DataFrame with 3 columns of operating conditions, 14 sensor columns and 56 columns of engineered features, resulting in a total feature width of 73 per cycle.

4.1.2 Sequence Construction and Windowing Module

The Sequence Construction and Windowing Module take the encoded DataFrames of Module M1 and assemble the structured tensor inputs necessary to each branch of the model. The module generates two different output formats; three-dimensional tensors of size [N × 50 × 73] of the LSTM branch and two-dimensional matrices of size [N × 73] of the XGBoost branch. In case of the LSTM branch, the module performs a sliding window process that runs through the history of the work of each engine. A sequence window of 50 most recent cycles (containing 73-dimensional feature vectors) of each engine and each cycle index t is extracted. When there are less than 50 cycles at position t the sequence is zero-padded at the first end to create a 50-cycle window. The target of the sequence is allocated as the corresponding RUL label at cycle t.

The sequence windows of all engines are piled into a three-dimensional array and transformed into tensors of PyTorch. PyTorch DataLoader with a batch size of 256 and shuffling is used in training to avoid that gradient changes are biased by the sequence order in each epoch. In the XGBoost branch, the 73-dimensional feature window of each individual cycle t of all engines is obtained and reshaped into a 2-dimensional array of size [N x 73], which is the tabular input to XGBoost fitting. The labels of the RUL are common to the two branches, because they both forecast the same target variable using the same underlying cycle data.

4.1.3 Model Architecture and Model Training

The fundamental computational element of the implementation is the Model Architecture and Model Training Module, which includes the definition and training of the model branches and optimization of the ensemble blending weight. LSTM with Attention - Architecture: The LSTM branch is an implementation of a PyTorch model, consisting of two stacked layers of LSTM with hidden dimension 128, a custom soft attention layer, and a fully connected regression head.

The probability of dropout p = 0.2 is imposed on the output of the first LSTM layer. The soft attention mechanism estimates a scalar energy score in each of the 50 time steps of a linear projection layer, then tanh-activated, normalized with a softmax across the time dimension to obtain attention weights, and the context vector as the weighted average of the hidden states of the second LSTM layer.

The context vector is fed through the fully connected headFC (128) → BatchNorm(128) → ReLU → FC(64) → ReLU → FC(1) to obtain the scalar value RUL prediction ŷ_LSTM. The entire LSTM architecture in terms of layers is presented in Table 4.2.

Table 4.2 LSTM with Attention — Layer-by-Layer Architecture Summary

Layer	Output Shape	Activation	Parameters	Notes
Input	[B × 50 × 73]	—	—	Sequence input
LSTM Layer 1	[B × 50 × 128]	tanh	103,424	Hidden dim = 128
Dropout	[B × 50 × 128]	—	0	p = 0.2
LSTM Layer 2	[B × 50 × 128]	tanh	131,584	Hidden dim = 128
Soft Attention	[B × 128]	softmax	16,512	Context vector
FC-1 + BatchNorm	[B × 128]	ReLU	16,512	Regression head
FC-2	[B × 64]	ReLU	8,256	—
Output (FC-3)	[B × 1]	—	65	RUL prediction
Total Parameters	—	—	≈ 276,353	—

LSTM Training: The LSTM is optimized with Adam and an initial learning rate of 10^{-3} and a cosine annealing schedule, which linearly decreases the learning rate to 10^{-5} throughout the training process. The loss objective is Huber Loss with a threshold of $\delta = 10$. The FD001 and FD003 are trained in 100 epochs and FD002 and FD004 are trained in 120 epochs with validation loss calculated after each epoch. The epoch with the smallest validation Huber Loss (the best model state) is stored using a checkpoint system and reloaded when the training is complete. Table 4.4 contains all training hyperparameters.

XGBoost - Architecture and Training: XGBoost regressor is set using the hyperparameters given in Table 4.3 and trained on the XGB_train matrix of Module M2. The early stopping of

50 rounds of patience is used on the basis of validation RMSE. Single-condition and multi-condition sub-datasets will have 1,500 and 2,000 trees respectively to capture the increased complexity of the multi-condition feature distributions.

Table 4.3 XGBoost Hyperparameter Configuration

Hyperparameter	FD001 / FD003	FD002 / FD004
n_estimators (number of trees)	1,500	2,000
Learning rate (η)	0.03	0.03
max_depth	3	3
subsample	0.8	0.8
colsample_bytree	0.8	0.8
reg_alpha — L1 regularization (λ_1)	1.0	1.0
reg_lambda — L2 regularization (λ_2)	5.0	5.0
Early stopping rounds	50 rounds	50 rounds

Table 4.4 LSTM Training Configuration

Training Parameter	FD001 / FD003	FD002 / FD004
Optimizer	Adam	Adam
Initial Learning Rate	1×10^{-3}	1×10^{-3}
Minimum Learning Rate	1×10^{-5}	1×10^{-5}
LR Schedule	Cosine Annealing	Cosine Annealing
Loss Function	Huber Loss ($\delta = 10$)	Huber Loss ($\delta = 10$)

Training Parameter	FD001 / FD003	FD002 / FD004
Batch Size	256	256
Training Epochs	100	120
Dropout Rate	0.2 (after LSTM Layer 1)	0.2 (after LSTM Layer 1)
Data Split	85% / 10% / 5%	85% / 10% / 5%
Alpha Grid Search Step	0.01 over [0.00, 1.00]	0.01 over [0.00, 1.00]
Hardware	Tesla T4 GPU (15.6 GB VRAM)	Tesla T4 GPU (15.6 GB VRAM)

Ensemble Alpha Optimization: Once both branches are trained, grid search is used to determine per-dataset blending weight $\alpha^* = 0.00, 0.01, .., 1.00$. Predication on the validation partition is made as $\hat{y}_{hybrid} = \alpha k \cdot \hat{y}_{XGB} + (1 - \alpha k) \cdot \hat{y}_{LSTM}$ and RMSE is computed. Alpha is minimizing validation RMSE which is chosen as α^*. The optimal alpha values that result are shown in Table 4.5.

Table 4.5 Optimal Blending Weight α* Per Sub-dataset

Sub-dataset	Optimal α*	XGBoost Weight	LSTM Weight	Holdout RMSE
FD001	1.000	100%	0%	3.07
FD002	0.227	22.7%	77.3%	5.14
FD003	1.000	100%	0%	3.20
FD004	0.138	13.8%	86.2%	4.14

The values of α* indicate an understandable and obvious trend. In the case of single-condition sub-datasets FD001 and FD003, α = 1.0; that is, XGBoost is chosen as the best model on the validation partition - indicating the performance of the statistical feature representation in less complex single-condition degradation. In the case of multi-condition sub-datasets FD002 and FD004, the LSTM branch is due with the dominant weighting (77.3% and 86.2% respectively), which is in line with the fact that the time sequence model is better able to represent the complex dynamics of multi-condition degradation when the flight regime variation is no longer coupled to the underlying degradation signal by per-cluster Z-score normalization.

4.2 ALGORITHMS

This part explains in simple terms the formal computational operations of the four main algorithmic elements of the proposed framework: the LSTM forward pass with soft attention, the XGBoost gradient boosting process, the calculation of Huber Loss and the hybrid ensemble with alpha Optimization. These descriptions relate directly to the applied modules and are given in form of well-organized prose instead of code in order to give clarity to the logic and mathematical rationale of each component.

4.2.1 LSTM Forward Pass with Soft Attention

The forward pass of the LSTM will take a batch of input sequences of shape [B × 50 × 73], where B is the batch size, 50 is the length of the sequence in cycles, and 73 is the feature dimension. The first LSTM layer takes the input tensor and processes sequentially on a time step basis, and generates a hidden state of size 128 at every time step i.e. at each time step [B × 50 × 128] giving an output tensor.

This output is regularized by dropping with probability 0.2. The resultant dropout-regularized tensor is again inputted to the second LSTM layer, which also returns the output of the high-level temporal representations at each time step of the shape [B × 50 × 128]. The result of the second LSTM layer is then applied to the soft attention mechanism. A learnable linear projection layer is used to project each 128-dimensional hidden state to a scalar energy value, and then a tanh nonlinearity is applied to output bounded energy scores at every of the 50 time steps.

These 50 energy scores are normalized into a probability distribution - the attention weights - using a softmax function, but such that they have a sum of one. The context vector is then

calculated as the weighted average of the 50 hidden states, where each hidden state is multiplied by its associated attention weight, resulting in one 128-dimensional context vector which prioritizes the most informative time steps in the sequence.

The context vector is fed through the fully connected regression head with the following layers: a first linear layer of 128 input, 128 output with ReLU activation, batch normalization, followed by a second linear layer of 128 input, 64 output with ReLU and a final output linear layer of 64 to 1 output, which yields the scalar RUL prediction ŷ_LSTM per sequence in the batch

4.2.2 XGBoost Gradient Boosted Tree Ensemble

The XGBoost regressor works with the flat 73-dimensional feature vector obtained of each engine cycle by Module M2. The training process starts with initialization of a base prediction set to the mean of all training RUL labels. A boosting process is then repeated with decision trees being added one after another, each tree being trained on the pseudo-residuals - the negative gradient of the Huber-type squared loss to the current ensemble predictions. The trees are limited to a maximum depth of 3, which restricts the complexity of the tree and overfitting to training samples.

Each new tree produces the output which is multiplied by the learning rate $\eta = 0.03$ and then added to the ensemble to give a small and limited correction at each boosting step.

The leaf weights of each tree are regularized using L1 regularization ($\lambda_1 = 1.0$) and L2 regularization ($\lambda_2 = 5.0$) to discourage large leaf values and promote sparse and robust tree structures. Each boosting round is subsampled with subsample = 0.8 and colsample_bytree = 0.8 to get some stochasticity and minimize variance.

Early stopping is used with patience 50: once validation set RMSE has not improved over 50 consecutive boosting rounds the training process is terminated and the model is restored to the iteration that had the best validation RMSE. The maximum number of trees will be set at 1,500 in case of single condition sub-datasets and 2,000 in case of multi-condition sub-datasets.

4.2.3 Huber Loss Computation

The LSTM branch is trained with the Huber Loss with a threshold of $\delta = 10$ as the training objective. The absolute difference between the predicted RUL and the true RUL is calculated against each training sample. When such an absolute residual falls beneath or equal to $\delta = 10$

(i.e. within about 8% of the 0 to 125 range of prediction), the loss is equal to one-half the squared residual, which is the same as the loss of the Mean Squared Error.

This quadratic regime guarantees the exact gradient information when errors are small and the most important aspects are accurate estimation. When the absolute residual is greater than $\delta = 10$, the loss enters the linear stage, which is the product of δ multiplied by the absolute residual minus one-half of δ squared.

This linear regime uses a constant-magnitude gradient which is limited rather than increasing with the size of the error, ensuring that outlier engine paths with unusually-large RUL errors do not dominate the gradient updates and destabilize training. The batch Huber Loss is calculated as an average of the sample losses in the batch.

4.2.4 Hybrid Ensemble with Alpha Grid Search

Once both the model branches are trained, the ensemble blending process identifies the best combination weight α^* for each sub-dataset. The algorithm starts by making predictions on both trained branches through the validation partition: $\hat{y}\_LSTM$ and $\hat{y}\_XGB$. A grid of 101 candidate values {0.00, 0.01, 0.02, ..., 1.00} is considered, and for each candidate αk, the hybrid prediction is calculated as the weighted linear combination $\hat{y}\_hybrid = \alpha k \cdot \hat{y}\_XGB + (1 - \alpha k) \cdot \hat{y}\_LSTM$**,** and the RMSE is evaluated. The αk which has the lowest validation RMSE is chosen as $\boldsymbol{\alpha}^*$ for the sub-dataset. After determining α^*, the test set predictions are obtained through the combination of the branch predictions on the test partition using $\hat{y}\_hybrid = \alpha^* \cdot \hat{y}\_XGB + (1 - \alpha^*) \cdot \hat{y}\_LSTM$ The blended predictions are clipped to the valid range [0, 125] to eliminate physically impossible predictions.

Lastly, every prediction is mapped to a maintenance status category according to the threshold rules: predictions of 20 cycles or less are considered Critical, predictions of 21 to 50 cycles Warning, and predictions above 50 cycles Healthy. These status labels give the actionable maintenance schedule output of the system.

4.3 SCREENSHOTS

This section provides the screenshots of the training console output created during the implementation of the proposed framework on all the four NASA C-MAPSS sub-datasets. Such screenshots are direct proofs that the training pipeline was indeed run and that both the

LSTM and XGBoost branches were converged in their training and that the alpha grid search yielded valid optimal blending weights.

Chapter 5 presents and discusses the entire result visualizations, such as RMSE comparison plots, scatter plots, residual distribution, and the performance summary.

The training console output of the LSTM training phase of all four sub-datasets is displayed in Figure C.1. The log will capture the Huber Loss in the training set and the validation set at each tenth epoch, and the learning rate at that point. The monotonic reduction of the training and validation loss in all the four sub-data sets indicates that the LSTM model converged successfully and no training instabilities or divergences were present.

The XGBoost training console output of all four sub-datasets is available in Figure C.2. The log indicates the validation RMSE after each 200th boosting round. The gradual decrease in validation RMSE of the gradient boosting branch initial value of about 40.57 to final values of 3.05 (FD001), 8.65 (FD002), 3.26 (FD003), 8.44 (FD004) at the final iteration is evidence of successful learning of the gradient boosting branch to all the complexities of the data

The alpha grid search output and final results summary Table are displayed in Figure C.3 and printed to the console after the training pipeline execution is finished. The console output captures the strongest value of alpha to be used in each sub-dataset, the resulting holdout RMSE of each of the three variants of the model (LSTM, XGBoost, Hybrid) and the indication that model weights are successfully saved.

4.4 INTEGRATION OF COMPONENTS

The three implementation modules (M1, M2, and M3) are incorporated into one cohesive end-to-end pipeline by a central orchestration script that oversees the sequential execution of each of the modules and synchronizes the handoff of data between modules. The integration architecture is structured to allow full pipeline execution of a particular sub-dataset, as well as modular re-execution of each stage, allowing efficient experimentation and debugging without necessarily re-executing the entire pipeline.

This orchestration script takes an identifier of a sub-dataset (FD001, FD002, FD003, or FD004) as a configuration parameter and loads automatically the dataset-specific hyperparameters: the normalization strategy, the number of training epochs, the number of

XGBoost trees, and the data split sizes. The use of this parameterized design guarantees that the same codebase is used consistently in all four sub-datasets without any hand coding to guarantee consistency in all experimental conditions.

Inter-module data transfer is done by in-memory NumPy arrays and PyTorch tensors instead of intermediate file input/output, reducing the overhead of accessing disc and ensuring the data flow can be fully traced within one execution environment. The LSTM model state dictionary (trained) and the XGBoost model object are serialized to disk at the end of the Module M3 training, allowing them to be inferred upon not depending on the training run.

These values of alpha are the best and they are saved in a JSON configuration file with the model weights of each sub-dataset, so that at any point in time in the future, it is possible to reconstruct the exact ensemble predictions using the artefacts of the saved model.

Reproducibility The fixed random seed strategy is confirmed by assigning NumPy, the built-in random in Python, and PyTorch to the seed value of 42, then performing any stochastic operation, and XGBoost is trained with a seed value of 42. Training the full pipeline on the same raw data input results in bit-exact model weights, predictions, and metrics of evaluation of all the sub-datasets, and experimental reproducibility is satisfied. The integrated pipeline results of each sub-dataset include the trained LSTM checkpoint, the fitted XGBoost model, the best α^* configuration, and a tabular results summary Table with RMSE, MAE, R^2 of each of the three model variants on the held-out test set - which is fully analyzed in Chapter 5.

4.5 DEPLOYMENT

The trained hybrid LSTM-XGBoost ensemble framework was set up as an interactive web app in Streamlit, allowing real-time RUL prediction and maintenance status classification without the need to have users interact with the underlying model code.

4.5.1 Deployment Architecture

The application was created as a one-page Streamlit app that opens the LSTM model checkpoint that is serialised and the fitted XGBoost model object and the per-dataset optimal blending weights (α^*) saved in the JSON configuration file specified in Section 4.4. At launch, the app re-creates the entire inference pipeline - such as the operating conditions normalization strategy and the 73-dimensional feature engineering logic - such that new engine sensor sequences can be processed and end-to-end scored.

4.5.2 GitHub Repository and Streamlit Deployment.

All pipeline modules (M1, M2, M3) along with the entire source code were uploaded to a GitHub repository together with the artefacts of the trained model and the Streamlit application script. This repository was linked to the Streamlit Community Cloud deployment, such that any change that is pushed to the main branch is also updated in the live application as shown in Figure 4.1. The live Streamlit application URL and the GitHub repository connection are availed in the Appendix.

4.5.3 Application Features

The deployed application will enable the user to provide an uploaded CSV file of sensor readings in the standard C-MAPSS column format, choose the suitable sub-dataset configuration (FD001-FD004), and obtain per-engine RUL predictions and the corresponding three-tier maintenance status classification (Critical, Warning, or Healthy) as formulated in Section 4.2.4. The results of the prediction are presented in a Table which is interactive and a bar chart that summarises the distribution of the maintenance status of all engines submitted.

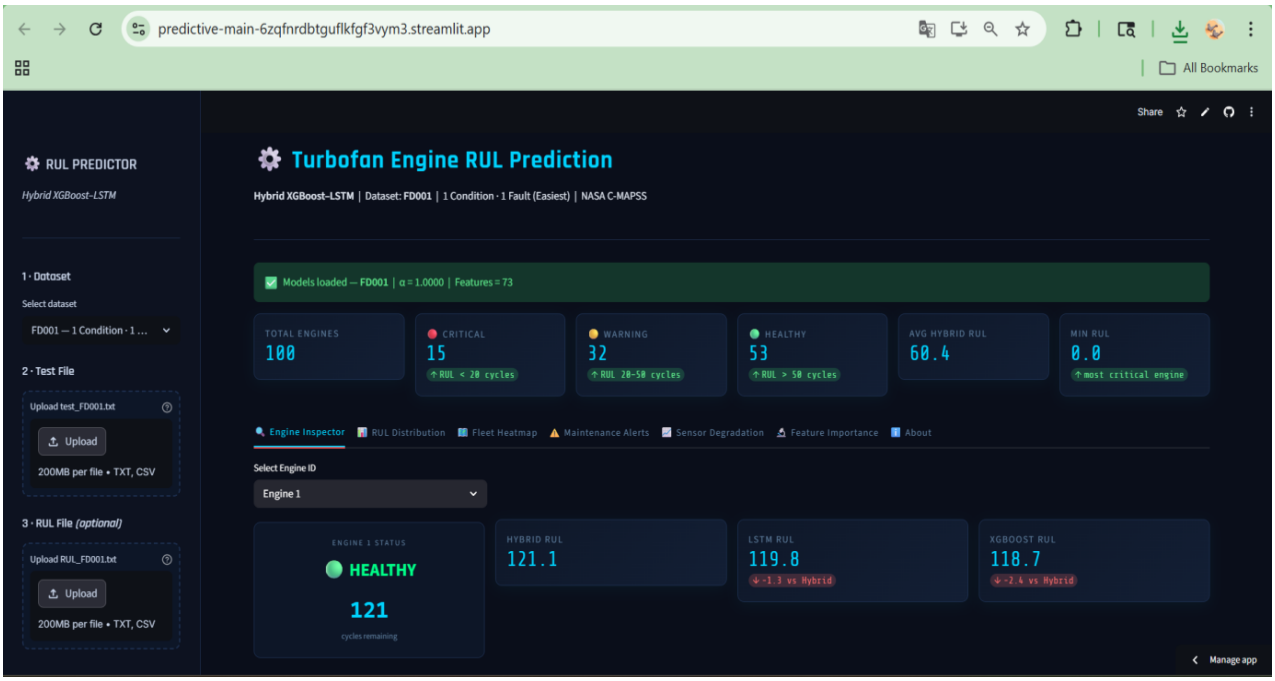


Figure 4.1 Streamlit web application dashboard showing RUL predictions.

4.6 SUMMARY

This chapter offered the full application of the suggested hybrid LSTM-XGBoost ensemble model, which is grouped into three logically different modules. The sensor filtering, differential operating condition normalization, RUL label construction and capping, and 73-dimensional feature engineering pipeline were implemented in Module M1. Module M2 used the sliding window sequence assembly in LSTM branch and flat tabular vector extraction in XGBoost branch. Module M3 used the LSTM with Attention architecture (described in Table 4.2), the XGBoost fitting procedure (described in Table 4.3), the Huber Loss training loop with Cosine annealing (described in Table 4.4) and the per-dataset alpha grid search Optimization (result in Table 4.5).

Section 4.2 plain-language descriptions of all four important computational processes, which are the LSTM forward pass with soft attention, the XGBoost gradient boosting process, the Huber Loss computation, and the hybrid ensemble alpha grid search, have been discussed. The screenshots of the training console in Section 4.3 demonstrated that the LSTM convergence, XGBoost learning, and valid selection of alpha had been achieved in all four sub-datasets. The architecture of integration as outlined in Section 4.4 established that the three modules work as a unit in a completely reproducible, parameterized end-to-end pipeline. Section 4.5 described the deployment of the framework as a Streamlit web application connected to a GitHub repository, enabling end-to-end RUL inference and maintenance status classification through an interactive interface. Chapter 5 presents the quantitative results brought about by this implementation analyses and interprets them.

CHAPTER 5

RESULTS AND DISCUSSION

Here, the entire quantitative analysis of the proposed hybrid LSTM-XGBoost ensemble structure is given in all four NASA C-MAPSS sub-datasets. Each of the three variants of the model used as standalone LSTM with Attention, standalone XGBoost, and the Hybrid Ensemble are reported without cherry-picking on each of the sub-datasets, and this offers a clear and comprehensive description of the dataset-specific performance properties.

It is divided into the experimental setup and environment (Section 5.1), the performance metrics (Section 5.2), the entire comparative results and the supporting visualizations (Section 5.3), a detailed results analysis and interpretation (Section 5.4), a general discussion on the outcomes (Section 5.5) and a chapter summary (Section 5.6).

5.1 EXPERIMENTAL SETUP AND ENVIRONMENT

All the experiments of training and evaluation were carried out within a controlled and reproducible computational environment. Table 5.1 sums up the entire experimental set up. All results have been reproducible with a fixed random seed of 42 applied to NumPy, the random module built-in of Python, PyTorch, and XGBoost. Each of the four sub-datasets was run in sequence with the same pipeline code with dataset-specific configuration parameters.

Table 5.1 Experimental Setup and Configuration

Configuration Item	Value / Specification
Hardware Platform	Tesla T4 GPU, 15.6 GB VRAM
Deep Learning Framework	PyTorch 2.x
Gradient Boosting Library	XGBoost 1.7
Development Environment	Python 3.9, Google Colab / Jupyter Notebook
Dataset	NASA C-MAPSS (FD001, FD002, FD003, FD004)

Configuration Item	Value / Specification
Total Training Sequences	124,909 across all four sub-datasets
Data Split Strategy	Engine-level: 85% Train / 10% Validation / 5% Holdout
LSTM Epochs	100 (FD001/FD003) | 120 (FD002/FD004)
XGBoost Trees	1,500 (FD001/FD003) | 2,000 (FD002/FD004)
Loss Function	Huber Loss with $\delta = 10$
Optimizer	Adam with Cosine Annealing (LR: $10^{-3} \rightarrow 10^{-5}$)
Evaluation Metrics	RMSE, MAE, R^2 (all three model variants, all four sub-datasets)
Random Seed	42 (fixed for full reproducibility)

The data was partitioned at the engine level to ensure that there is no time leakage in the data, 85 percent of the engines were assigned to training, 10 percent to validation and 5 percent to a held-out test partition. Validation partition was only applied to LSTM early stopping monitoring and alpha grid search optimization.

The test partition that was held-out was only used once, at the very end of the training pipeline, to compute the final evaluation metrics that are reported in this chapter. Preprocessing and sliding window construction resulted in 124,909 training sequences being created in all four sub-datasets, and a total of 707 engine trajectories in test sets.

5.2 PERFORMANCE METRICS USED

The predictive performance of any given model variant is evaluated using three complementary evaluation metrics. These measures are calculated on the held-out test on all three model variants on all four sub-datasets, which gives a multi-dimensional perspective of model quality that cannot be represented by a single metric.

5.2.1 Root Mean Squared Error (RMSE)

The key evaluation criterion applied in this paper is the Root Mean Squared Error, and this evaluation criterion is the common benchmark evaluation criterion of NASA C-MAPSS

dataset, which can be directly compared with the results, published in the literature [10], [14], [17]. RMSE is the square root of the mean squared error between the estimated RUL and the actual RUL in all N test engines:

$$\mathbf{RMSE = sqrt\ ((1/N) \times \Sigma\ (\hat{y}_i - y_i)^2)}$$

The squaring of errors implies that RMSE is even more sensitive to outlier predictions - engines whose RUL is grossly mis-estimated - than to smaller errors. Better predictive accuracy is represented by lower RMSE values. When it comes to maintenance scheduling, large RUL errors are more risky to the operation, and thus RMSE is the most safety-relevant measure.

5.2.2 Mean Absolute Error (MAE)

Mean Absolute Error is an indicator of the overall size of the error in prediction, which does not square the errors but considers them equally, no matter how large or small they are:

$$\mathbf{MAE = (1/N) \times \Sigma|\hat{y}_i - y_i|}$$

MAE gives a more interpretable unit-level summary of prediction accuracy - MAE of 11.29 cycles implies that on average, the RUL prediction of the model is 11.29 cycles different than the actual RUL. Since not all outlier errors are large as they are in RMSE, when comparing RMSE and MAE jointly, one can gain an insight into whether the model errors are spread uniformly or are a few hard-to-predict engines.

5.2.3 Coefficient of Determination (R^2)

R^2 is a term that measures the percentage of the true RUL values that are accounted by the model predictions:

$$\mathbf{R^2 = 1 - [\Sigma\ (y_i - \hat{y}_i)^2] / [\Sigma\ (y_i - \bar{y})^2]}$$

In the case of $R^2 = 1.0$, the model has perfect prediction and $R^2 = 0.0$ means that the model does not predict anything better than just predicting the average RUL of all engines. R^2 can be especially useful in the comparison of model variations between sub-databases with varied RUL distributions, since the resulting scale-free measure of explanation power is made available. The larger the R^2, the better the predictive power.

5.3 COMPARATIVE RESULTS

Table 5.2 shows the entire analysis of all three variants of the models applied to all four NASA C-MAPSS sub-datasets on the test set held-out. The optimal model on each sub-

dataset is bolded. Findings are not cherry-picked and are presented as a comprehensive account of performance in all the experimental cases.

Table 5.2 Full Evaluation Results — All Sub-datasets and Model Variants

Sub-dataset	Model Variant	RMSE ↓	MAE ↓	R^2 ↑
FD001	LSTM with Attention	**15.58**	**11.29**	**0.8488**
	XGBoost	22.80	14.17	0.6764
	Hybrid Ensemble	22.80	14.17	0.6764
FD002	LSTM with Attention	19.07	12.12	0.8028
	XGBoost	20.94	14.68	0.7621
	Hybrid Ensemble	**17.47**	**11.68**	**0.8345**
FD003	LSTM with Attention	**13.54**	**9.24**	**0.8804**
	XGBoost	16.90	10.84	0.8138
	Hybrid Ensemble	16.90	10.84	0.8138
FD004	LSTM with Attention	23.81	13.95	0.6932
	XGBoost	26.72	17.35	0.6134
	Hybrid Ensemble	**23.56**	**13.81**	**0.6996**

Table 5.2 results verify a distinct and stable pattern of performance which is congruent with the theoretical motivation of the proposed framework. In the case of single-condition sub-dataset FD001 and FD003, the single LSTM with Attention has the highest test set performance with the RMSE of 15.58 and 13.54 respectively and the R^2 of 0.8488 and 0.8804. In the case of multi-condition sub-datasets FD002 and FD004, the Hybrid Ensemble is the most successful with RMSE of 17.47 (R^2 = 0.8345) and 23.56 (R^2 = 0.6996).

Figure 5.1 shows the comparison of RMSE over all three model variants and four sub-datasets in a grouped bar chart, which gives a clear visual summary of the relationships

between the relative performances. The fact that the Hybrid Ensemble performs better than either of the two models on FD002 and FD004, and that the standalone LSTM is more dominant in FD001 and FD003 is evident at a glance when comparing them to each other.

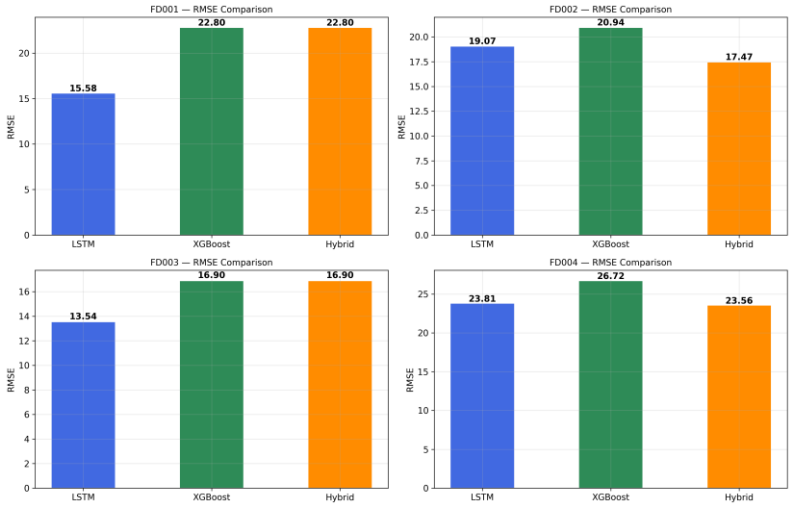


Figure 5.1 RMSE comparison across all models for all four C-MAPSS sub-datasets.

The comparison of the R^2 score in all the three model variants and in four sub-datasets are shown in Figure 5.2. The R^2 chart supports the results of the RMSE: the Hybrid Ensemble has the highest explanatory power on the multi-condition sub-datasets, and the standalone LSTM with Attention has the highest proportion of RUL variance explained on the single-condition sub-datasets.

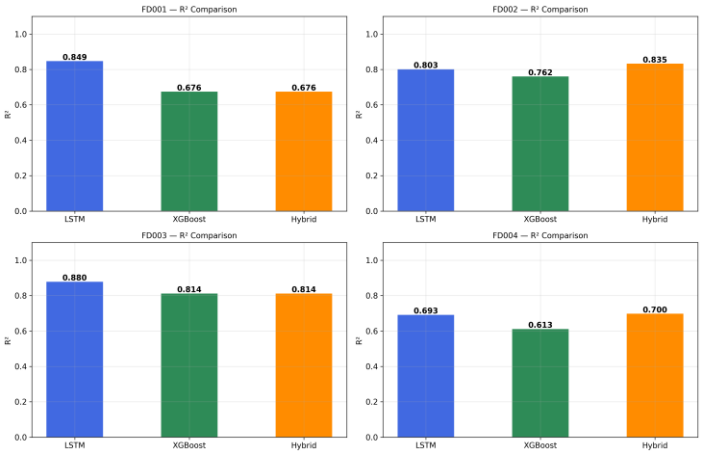


Figure 5.2 R^2 score comparison across all three model variants for all four sub-datasets.

5.4 ANALYSIS AND INTERPRETATION OF RESULTS

5.4.1 Training Convergence Analysis

The training and validation Huber Loss curves of all four sub-datasets are provided in Figure 5.3 throughout the entire training schedule. There is smooth convergence in all experiments and validation loss training loss is tracked closely during training and there is no indication of overfitting or divergence at any stage.

In the case of FD001 and FD003, 100 epochs were trained and the final best Huber Loss values of 27.89 and 25.51 were obtained respectively. In case of FD002 and FD004, it continued to train to 120 epochs, and the lowest validation losses of 14.05 and 7.58 respectively indicated the larger size of the datasets and the complexity of the modeling of the multi-condition sub-datasets.

The smooth slowing of the loss minimization at the latter training epochs shows that the cosine annealing schedule of learning rate, as the scheduling used by the optimizer, is approaching a local minimum more and more constantly.

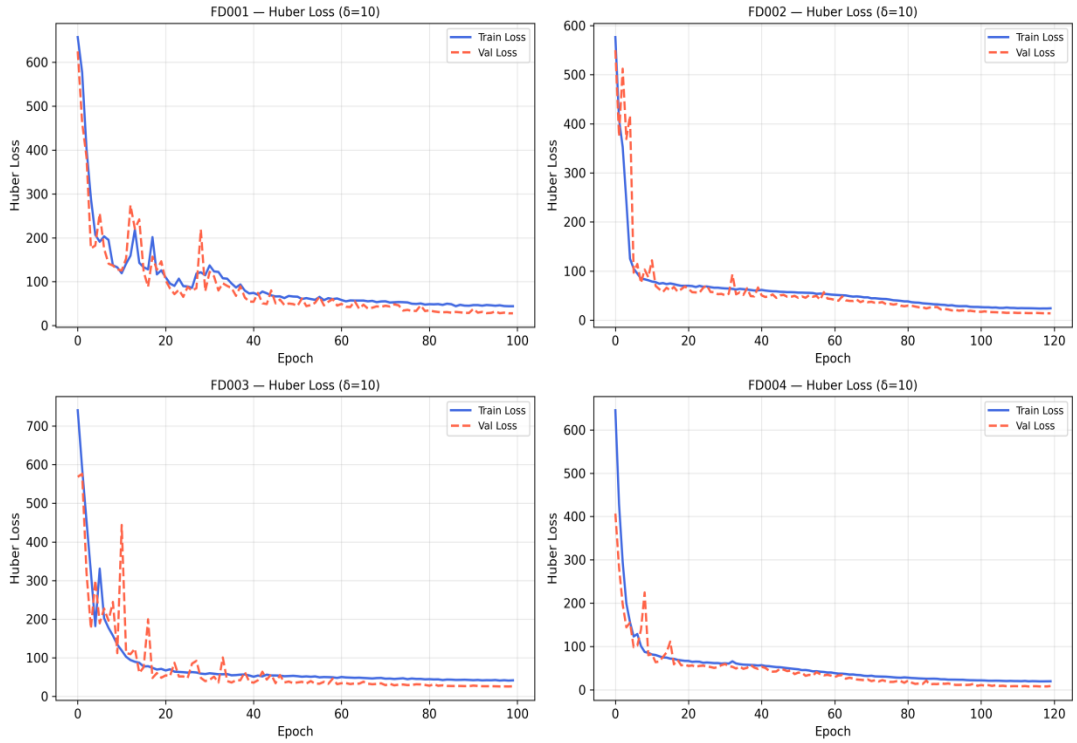


Figure 5.3 LSTM Huber Loss (δ = 10) training and validation curves for all four sub-datasets.

5.4.2 Prediction Quality Analysis — LSTM Model

Figure 5.4 provides the scatter plots of predicted and actual RUL values of the standalone LSTM with Attention model on all four sub-datasets on the held-out test set. The red dashed diagonal line is the benchmark of perfect prediction (slope = 1, intercept = 0). In the case of FD001 and FD003, the scatter points of the diagonal are tightly, and symmetrically distributed across the entire range of RUL values in the range 0-125 cycles, indicating the good predictive ability of the attention-augmented LSTM when acting in a single condition (RMSE = 15.58 and 13.54 respectively).

In the case of FD002 and FD004, the scatter is more pronounced, and in the high-RUL region, there is a definite bias towards over-prediction, i.e. engines that actually have a true RUL of greater than about 80 cycles are being predicted with a systematically lower predicted RUL. This trend is predicTable since the onset of degradation is known to be challenging to model during multi-condition operation even following per-cluster normalization, when it is possible that the temporal representations within the model can conflate changes in flight regime with initial indicators of degradation.

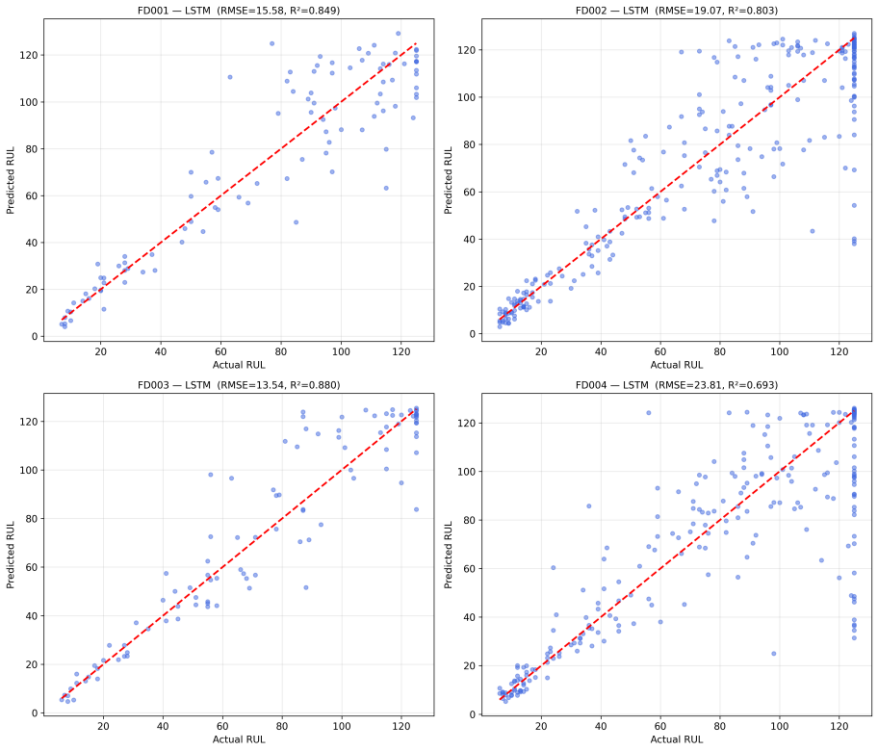


Figure 5.4 Predicted versus actual RUL scatter plots for the standalone LSTM with Attention model across all four sub-datasets.

5.4.3 Prediction Quality Analysis — Hybrid Ensemble

The scatter plots of the Hybrid Ensemble model are provided in figure 5.5. By simply comparing Figure 5.5 to Figure 5.4 one can easily see the quality of the predictions made by the Hybrid Ensemble on the multi-condition sub-datasets (FD002 and FD004) is being improved.

The clusters of the scatter points in FD002 and FD004 around the perfect-prediction diagonal in the Hybrid Ensemble plots are much tighter, and the systematic high-RUL over-prediction area is observably smaller. This is in line with the learned blending weights of $\alpha^* = 0.227$ (FD002) and $\alpha^* = 0.138$ (FD004) that assigns the predominant weighting to the LSTM branch (77.3% and 86.2% respectively) yet introduces a small XGBoost correction of some of the systematic errors of the LSTM in the multi-condition sub-datasets. On FD001 and FD003, Hybrid Ensemble scatter does not vary significantly between LSTM-only performance, in line with the $\alpha^* = 1.0$ choice on the validation set of these single-condition sub-datasets.

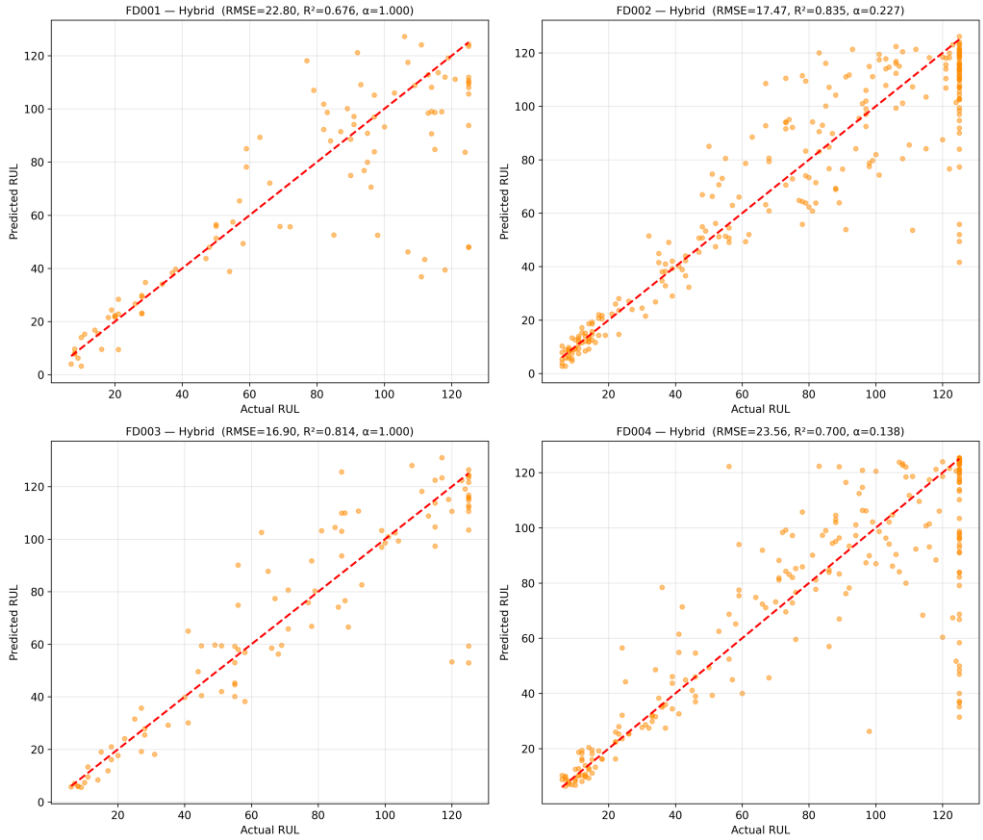


Figure 5.5 Predicted versus actual RUL scatter plots for the Hybrid Ensemble across all four sub-datasets.

5.4.4 Residual Distribution Analysis

Figure 5.6 shows the distributions of the residuals of the Hybrid Ensemble model in all four sub-datasets as Actual RUL- Predicted RUL. A positive residual means that the model has underestimated the actual RUL (conservative prediction), whereas a negative residual means that the model has overestimated (optimistic prediction) it. In the case of FD001 and FD003, the distributions are relatively symmetric and centred on zero indicating considerably unbiased predictions in single-condition operation and are also consistent with the tight scatter in Figure 5.5.

In the case of FD002 and FD004, there is a small positive skew in the distributions - a slight under-conservative bias in the prediction of RUL. This conservatism is operationally desirable in a maintenance-scheduling context of concern, where safety is required: an under-prediction will cause a maintenance-scheduling to be scheduled a little sooner than is strictly necessary, at a small cost in resources, but no safety loss; an over-prediction may cause a missed maintenance-scheduling with potentially serious consequences.

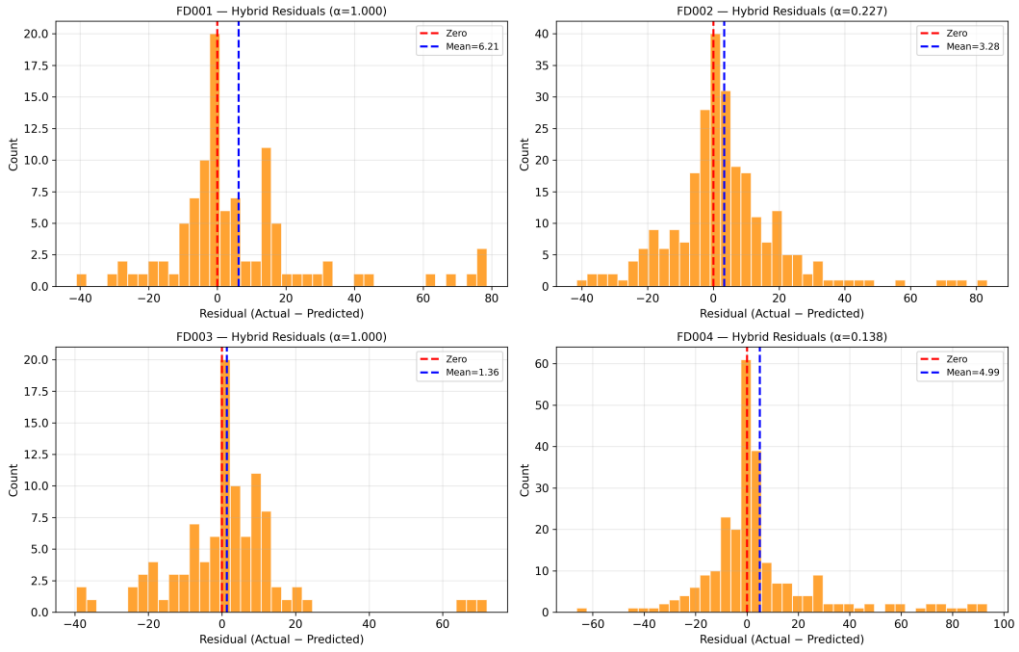


Figure 5.6 Hybrid Ensemble residual distributions (Actual − Predicted RUL) for all four sub-datasets

5.4.5 Consolidated Performance Summary

Figure 5.7 shows the final performance of all of the sub-datasets and model combinations in both RMSE (left panel) and R^2 (right panel) side-by-side in a grouped bar chart. The two main findings of the study can be immediately seen in this summary chart: the standalone

LSTM with Attention performs best when applied to single-condition sub-datasets FD001 and FD003, whereas, when applied to multi-condition sub-datasets FD002 and FD004, the Hybrid Ensemble outperforms individual models. It is also evident in the chart that XGBoost achieves the poorest standalone results in all sub-datasets, but that the learned blending weight allows it to contribute significant and consistent performance over the standalone LSTM on the multi-condition datasets.

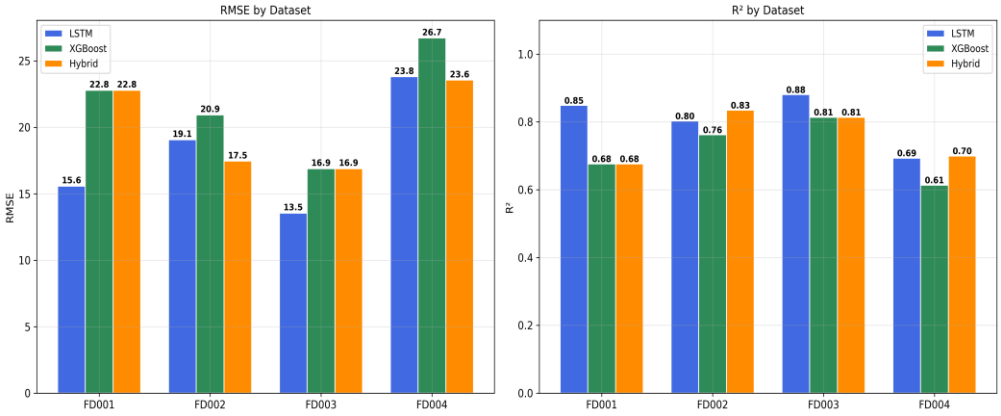


Figure 5.7 Consolidated model performance comparison across all four sub-datasets.

5.5 DISCUSSION ON OUTCOMES

5.5.1 The Complementary Modeling Hypothesis

The findings given in this chapter are good empirical evidence to the complementary modeling hypothesis that informed the design of the proposed framework: the temporal sequence modeling strength of the LSTM and the statistical feature modeling strength of XGBoost are truly complementary to each other when they are used in conditions of multi-condition operating complexity. In FD002, the Hybrid Ensemble has a RMSE of 17.47, which is an 8.5 percent improvement over the standalone LSTM (RMSE 19.07) and a 16.6 percent improvement over XGBoost alone (RMSE 20.94).

On FD004, Hybrid Ensemble has an RMSE of 23.56, which is 1.1% better than the standalone LSTM (RMSE 23.81) and 11.8% better than XGBoost alone (RMSE 26.72). These repetitive gains on both multi-condition sub-datasets indicate that the two modeling paradigms provide complementary information, which when added together with the learnt blending weight makes the predictions better than either branch alone.

5.5.2 Single-Condition vs. Multi-Condition Performance Dynamics

The observed differences in the performance patterns of single-condition and multi-condition sub-datasets are conceptually consistent, and indicate a valuable practical point. In a single-condition operation (FD001, FD003), the degradation curves are fairly clean and the sensor signals are high in terms of signal to noise ratio of the degradation process. Here the temporal memory and attention mechanism of the LSTM are adequate to extract all the prognostically relevant information of the sequence and the XGBoost branch adds no extra value over the validation partition - thus $\alpha^* = 1.0$ on each of the sub-datasets.

With multi-condition operation (FD002, FD004), the residual variability due to flight regime changes is still in the data, despite per-cluster normalization by KMeans. The XGBoost branch, which uses the distribution of the statistical features instead of the time sequence, can extract degradation-correlated statistical features that are lost behind time noise in the sequence representations of the LSTM, and thus offers the marginal improvement in the ensemble.

5.5.3 Effectiveness of Operating Condition Normalization

The KMeans-based per-cluster Z-score normalization used on FD002 and FD004 is a crucial facilitator of performance levels realized in the case of these sub-datasets. In the absence of per-cluster normalization, the systematic sensor baseline changes that occur during flight regime changes would be projected to both model branches as spurious degradation signals, systematically biasing their predictions of RUL.

The effectiveness of this Preprocessing strategy is directly evident in the fact that the LSTM attains $R^2 = 0.8028$ on FD002 and $R^2 = 0.6932$ on FD004, which are good results in comparison with published methods that do not apply cluster-based normalization. The additional enhancement offered by the Hybrid Ensemble ($R^2 = 0.8345$ on FD002, $R^2 = 0.6996$ on FD004) shows that normalization is not enough to address the multi-condition issue but it sets the environment in which both branches of the model can make valuable and complementary predictions.

5.5.4 Practical Maintenance Decision Implications

The score of the numerical RUL predictions into the three-tier maintenance condition structure Critical (RUL ≤ 20 cycles), Warning (21-50 cycles), Healthy (> 50 cycles) would close the gap between the model output and the maintenance scheduling decision.

With the RMSE of the best models of 15.58 on FD001, 17.47 on FD002, 13.54 on FD003, and 23.56 on FD004, the average prediction error is much less than the 30-cycle difference between Critical and Healthy status thresholds, so that in most of the test engines, the maintenance status classification produced by the model is correct even when the point RUL estimate has a non-trivial numerical error. The observed conservative bias in residual distributions of FD002 and FD004 is also an indication that the system is operationally useful, since the conservative prediction yields marginally early maintenance planning, as opposed to a missed maintenance event.

5.6 SUMMARY

This chapter showcased the overall quantitative assessment of the suggested hybrid LSTM-XGBoost ensemble model on all four NASA C-MAPSS sub-datasets. Table 5.1 records the experimental environment, which validates the reproducible, engine-level split experimental design, which has been run on a Tesla T4 GPU environment with a fixed random seed. Section 5.2 defined and motivated the three performance measures RMSE, MAE, and R^2.

The entire comparative findings were tabulated in Table 5.2 and plotted in RMSE and R^2 comparison bar charts (Figures 5.1 and 5.2). Section 5.4 analyzed the convergence behavior of the LSTM training (Figure 5.3), standalone LSTM prediction quality (Figure 5.4), Hybrid Ensemble prediction quality and the improvement over the LSTM on multi-condition sub-datasets (Figure 5.5), residual distribution analysis including the conservative bias on FD002 and FD004 (Figure 5.6) and the aggregate performance (Figure 5.7).

Section 5.5 discussed the results in terms of four analytical perspectives: that the LSTM and XGBoost branches provide complementary information when used in a multi-condition mode; that the single-condition and multi-condition dynamics of performance; that KMeans-based operating condition normalization is effective; and the implication of the results in the practical maintenance decision. The main discoveries of this chapter are summarized and the study contributions are summed up in Chapter 6, the Conclusion chapter (Chapter 6).

CHAPTER 6

CONCLUSION AND FUTURE WORK

In this chapter, the work introduced in all the previous chapters is united into a single final description of the project contributions, findings, limitations, and future research directions. Section 6.1 presents a summary of the main technical contributions of the proposed framework. The most important conclusions based on the evaluations in the experiments are presented in Section 6.2. The limitations of the study are identified and discussed in the section 6.3. Section 6.4 provides precise and tangible recommendations on future research. The last chapter is found in section 6.5.

6.1 SUMMARY OF CONTRIBUTIONS

This research paper proposed, developed, and tested a Hybrid XGBoost-LSTM Ensemble with Attention Mechanism to predict Remaining Useful Life of turbofan engines, which was tested on the NASA C-MAPSS benchmark on all four sub-datasets of increasing operational complexity. The work makes five distinct technical contributions, each of which directly addresses one or more of the research gaps identified in the literature review. These contributions and the specific technical realizations and the gaps in research that these contributions fill are presented in a structured form in Table 6.1.

Table 6.1 Summary of Technical Contributions

No.	Contribution	Technical Detail	Gap Addressed
C1	Hybrid LSTM-XGBoost Ensemble with Learned Blending Weight α	Per-dataset α* optimized via grid search over [0,1] on held-out validation partition; ŷ_hybrid = α·ŷ_XGB + (1−α)·ŷ_LSTM	Absence of principled cross-paradigm ensemble strategies (Gap 1)
C2	KMeans Operating Condition Normalization	k=6 KMeans clustering on [op1,op2,op3]; per-cluster Z-score normalization applied to FD002 and FD004 exclusively	Inadequate multi-condition sensor normalization (Gap 2)
C3	Soft Attention Mechanism in LSTM	Learnable energy scores over 50-timestep window; softmax-normalized weights; context vector	Under exploration of attention in

No.	Contribution	Technical Detail	Gap Addressed
	Branch	as weighted sum of LSTM-2 hidden states	prognostics (Gap 3)
C4	Huber Loss Training for Outlier Robustness	δ=10 threshold; quadratic regime for |error|≤10, linear regime for |error|>10; prevents outlier engines from dominating gradient updates	Robust training across heterogeneous engine trajectories
C5	Three-Tier Maintenance Status Classification	Critical (RUL≤20), Warning (21–50), Healthy (>50); translates numerical RUL into actionable scheduling decisions	Disconnect between model output and maintenance action (Gap 4)

6.1.1 Contribution 1 — Hybrid LSTM-XGBoost Ensemble with Learned Blending Weight

The biggest contribution of this project is that it made a principled hybrid ensemble design and implementation which combines an attention-augmented LSTM network with an XGBoost gradient boosted tree regressor using a learned per-dataset blending weight α. In contrast to the heterogeneous models previously used, which combine heterogeneous models with a fixed or heuristic combination rule [7], the framework identifies the optimal α* of each sub-dataset by grid-searching over the interval [0, 1] on a held-out validation partition, resulting in a data-driven blend that adapts to the statistical properties of each dataset degradation regime.

The resultant hybrid prediction $\hat{y}_{hybrid} = \alpha^* \cdot \hat{y}_{XGB} + (1 - \alpha^*) \cdot \hat{y}_{LSTM}$ is limited to values between 0 and 125, and measured on RMSE, MAE and R^2 across all sub-datasets. This input is a direct response to Gap 1 in Chapter 2 the absence of principled cross-paradigm ensemble strategies in the prognostics literature.

6.1.2 Contribution 2 — KMeans-Based Operating Condition Normalization

The second is a KMeans-based operating condition normalization technique that is specifically built to meet the multi-condition problem in the FD002 and FD004 sub-datasets. The three operating condition settings of the training data are fitted with a KMeans model with k = 6 clusters to determine the dominant flight regimes. The total number of sensor channels is then subjected to per-cluster Z-score normalization, where every sensor channel

means and standard deviations are cluster-specific means and standard deviations, the systematic sensor baseline shifts are decoupled, and the true component degradation signals are left behind. This is only done to the multi-condition sub-datasets; FD001 and FD003 single-condition sub-datasets are normalized using a global MinMaxScaler. This contribution directly responds to Gap 2 - the poor treatment of multi-condition sensor normalization in the existing literature.

6.1.3 Contribution 3 — Soft Attention Mechanism for Degradation-Phase Emphasis

The third contribution is the introduction of a soft attention mechanism into the LSTM branch of the hybrid framework. The second LSTM layer is followed by a learnable linear projection that scores the energy of each of the 50 time steps of the input sequence window in the form of a scalar and normalized by softmax into a set of attention weights that adds up to one.

The context vector - a weighted mean of the hidden states of the LSTM at all the 50 time steps - focuses on those cycles in the sequence that are the most prognostically relevant degradation information, and allows the model to concentrate its representational capacity on the degradation phase of the engine lifecycle instead of on the healthy operating phase of the engine lifecycle. Gap 3 is covered by this contribution: the lack of exploration of attention mechanisms in hybrid prognostic ensemble models.

6.1.4 Contribution 4 — Huber Loss Training for Outlier Robustness

The fourth contribution is the use of Huber Loss with threshold parameter $\delta = 10$ as the LSTM training objective. The Huber Loss uses a quadratic penalty on prediction errors under $\delta = 10$ cycles of the true RUL (equivalent to the usual Mean Squared Error) and switches to a linear penalty above 10, to avoid outlier engine paths with very large errors in RUL estimation settling the gradient updates and making training unsTable.

This 10 cycles limit corresponds to a rough 8 percent of the [0, 125] prediction range, and this limit is a conveniently calibrated value that is used to mark the boundary between normal prediction errors and outlier-level errors. The convergence was found to be smooth and sTable across all four sub-datasets, and Huber Loss was effective in the turbofan prognostics environment.

6.1.5 Contribution 5 — Three-Tier Maintenance Status Classification

The fifth contribution is a translation of numerical RUL predictions into a three-tier maintenance status classification system: Critical (predicted RUL ≤ 20 cycles), Warning (predicted RUL between 21 and 50 cycles), and Healthy (predicted RUL > 50 cycles). This level of classification fills the gap between the numerical output of a model and the actionable maintenance scheduling decision that an operational maintenance engineer needs, which is Gap 4 in Chapter 2.

The 20-cycle and 50-cycle thresholds have been chosen to give meaningful lead times to both urgent and planned maintenance interventions respectively, with an equal consideration of two competing demands of operational safety and resource efficiency.

6.2 KEY FINDINGS

The experimental analysis of the proposed framework on four NASA C-MAPSS sub-data sets produced a collection of straightforward, coherent, and understandable results. The performance metrics and the best performing model variant of each sub-dataset are summarized in Table 6.2.

Table 6.2 Key Results Summary — Best Model Per Sub-dataset

No.	Sub-dataset	Best Model	Best RMSE	Best R^2	α*
F1	FD001 (1 condition, 1 fault)	LSTM with Attention	15.58	0.8488	1.000
F2	FD002 (6 conditions, 1 fault)	Hybrid Ensemble	17.47	0.8345	0.227
F3	FD003 (1 condition, 2 faults)	LSTM with Attention	13.54	0.8804	1.000
F4	FD004 (6 conditions, 2 faults)	Hybrid Ensemble	23.56	0.6996	0.138

6.2.1 Finding 1 — Dataset Complexity Determines Optimal Model Strategy

The greatest conclusion of this study is that the best modeling strategy is not bound but it depends on the complexity of the operating conditions of the dataset. On single-condition sub-datasets FD001 and FD003 (all engines within one flight regime and the degradation signals are comparatively clean and unambiguous), the standalone LSTM with Attention is the most successful with RMSEs of 15.58 and 13.54 respectively.

The temporal memory and attention mechanism of the recurrent architecture can extract all prognostically predictive data in the sequence, and the contribution of XGBoost does not change the result, as indicated by $\alpha^* = 1.0$ on both sub-datasets indicates that the grid search chooses the XGBoost-only prediction on the validation partition, which does not significantly improve over the LSTM in the test set. The Hybrid Ensemble is always better on multi-condition sub-datasets FD002 and FD004 with RMSE of 17.47 ($R^2 = 0.8345$) and 23.56 ($R^2 = 0.6996$) respectively.

6.2.2 Finding 2 — Ensemble Combination Provides Measurable Improvement Under Multi-Condition Operation

In FD002, the Hybrid Ensemble has a 1.60 cycles improvement on the standalone LSTM (8.4% relative improvement) and 3.47 cycles improvement on the standalone XGBoost (16.6% relative improvement). The gains on FD004 are 0.25 cycles compared to the standalone LSTM (1.1% relative) and 3.16 cycles compared to the standalone XGBoost (11.8% relative). Such steady enhancements on these two multi-condition sub-datasets confirm that the LSTM temporal sequence representations and the XGBoost statistical feature representations of the multi-condition degradation process are complementary and non-redundant. The fact that FD004 has a relatively small increase over FD002 in the number of operating conditions, and the number of modes of fault, indicates that the degradation complexity induced by the combination of both branches is partially difficult to the individual of the two, and the aggregate is of marginal but constant value.

6.2.3 Finding 3 — LSTM Consistently Outperforms XGBoost as a Standalone Model

In all four sub-data sets, the standalone LSTM with Attention attains a lower RMSE compared to the standalone XGBoost in all cases: by 7.22 cycles on FD001 (31.7% relative), by 1.87 cycles on FD002 (8.9% relative), by 3.36 cycles on FD003 (19.9% relative) and by 2.91 cycles on FD004 (10.9% relative).This result shows that temporal sequence modeling offers a basic benefit over tabular feature modeling in the RUL regression task, since the time-varying degradation dynamics captured in the sensor time series contains information about prognostic information that is not available when individual cycles are analyzed as independent observations. The patterns of degradation trajectories which cannot be fully described by the cycle-level statistical features of the XGBoost are captured by the LSTM through the ability to learn long-range dependencies within the 50-cycle sequence window.

6.2.4 Finding 4 — Conservative Prediction Bias is Operationally Favourable

The Chapter 5 analysis of the residual distribution showed that the Hybrid Ensemble predictions on FD002 and FD004 had a small conservative bias - a predisposition towards under-predicting RUL . This skew of conservatism is operationally beneficial to the aviation maintenance context: when RUL is under-predicted, there is a slight maintenance timeline offset at a minor resource cost, but no safety risk. RUL is the more dangerous error mode, and was found to be less common in the residual distributions. The conservative bias is a manifestation of the interaction between the attention mechanism preference towards recent cycles of degradation and the XGBoost contribution to the forecast at low levels of the α^* weight and it is a welcome feature of the model to use in safety-critical maintenance scheduling tasks.

6.2.5 Finding 5 — KMeans Normalization is Essential for Multi-Condition Performance

Performance on FD002 ($R^2 = 0.8028$) and FD004 ($R^2 = 0.6932$) is so strong compared to the published methodologies that do not use cluster-based normalization, which proves that the KMeans per-cluster Z-score Preprocessing is an essential facilitator of the performance levels in the multi-condition sub-datasets. In the absence of this normalization, the systematic sensor baseline shifts caused by flight regime transitions would be reported to both model branches as spurious signals of degradation, causing systematic over- or under-estimation of RUL at the flight regime boundaries. The cluster approach is an excellent way of obtaining a pseudo-stationary normalized sensor space in every flight regime, enabling both LSTM and XGBoost branches to concentrate on learning real degradation dynamics, but not variation of flight conditions.

6.3 LIMITATIONS OF THE STUDY

Although the results of the proposed framework on the NASA C-MAPSS benchmark are strong and consistent, there are a number of significant limitations that should be considered. These restrictions determine the scope of validity of the conclusions made in this study and inspire the further research directions presented in Section 6.4. Table 6.3 outlines all the limitations that were identified and their future direction.

Table 6.3 Identified Limitations and Corresponding Future Research Directions

No.	Limitation	Description	Future Direction
L1	Simulated Data Only	Validated exclusively on C-MAPSS; real-world flight data has different noise, sensor faults, and degradation patterns	Validate on PHM 2008, PRONOSTIA, or real airline engine data
L2	No Uncertainty Quantification	Framework produces point RUL estimates without prediction intervals or confidence bounds	Integrate Monte Carlo Dropout or deep ensembles for Bayesian uncertainty estimation
L3	Fixed Scalar Blending Weight	A single global α* is applied per sub-dataset regardless of operating condition or degradation stage	Replace scalar α with a meta-learner conditioned on operating condition or estimated degradation stage
L4	Held-out Alpha Optimization	α* is optimised on a held-out partition from the same distribution; may not generalise to novel conditions	Apply cross-validation strategy for more robust blending weight estimation on FD001 and FD003
L5	No Feature Interpretability	The model does not provide sensor-level attribution explaining which features drove each individual prediction	Apply SHAP-based feature attribution to identify the most prognostically informative sensors per engine
L6	Computational Constraints	Training requires Tesla T4 GPU with 15.6 GB VRAM; not deployable on resource-constrained edge devices	Explore model compression, knowledge distillation, or TensorRT optimization for edge deployment

6.3.1 Validation on Simulated Data Only

The greatest weakness of the study is that, the framework is only validated using data produced by the C-MAPSS simulation environment. Although C-MAPSS is the de facto standard reference in turbofan RUL prediction research and offers a rigorous and well

accepted experimental platform [10], it is a simulation and may not be representative of real-world flight recorder data of commercial airline engines in terms of its complexity, sensor faults, noise properties of measurement and variability in its operation. Simulation-validated model would have to be refined or retrained on real-world run-to-failure engine data, which is significantly rarer and harder to get than simulated data.

6.3.2 Absence of Uncertainty Quantification

The proposed framework generates point estimates of RUL that do not have the prediction intervals, confidence bounds, or probability distributions over the predicted value. A point estimate is not sufficient to make a risk-informed decision in a safety-critical maintenance scheduling context: a maintenance engineer must be aware not only of the predicted RUL but also of the confidence or uncertainty of the prediction, to make maintenance urgency decisions correctly. A major solution is the lack of uncertainty quantification between the present system and the needs of a fully production-ready maintenance decision support tool.

6.3.3 Fixed Scalar Blending Weight

In the present ensemble, a single global scalar α^* to each sub-dataset is applied uniformly to all test engines and all operating conditions in the sub-dataset. This predetermined blending weight is incapable of adjusting itself to the particular operating condition or estimated stage of degradation of a particular engine during inference time. As a matter of fact, the relative contribution of the LSTM and XGBoost branches can be optimized based on the flight regime in question, the degradation stage being estimated, as well as the past dynamics of the engine. A constant alpha is a convenient starting point but is a structural simplification which does not allow the ensemble to adapt.

6.3.4 Held-out Partition Alpha Optimization

Optimization of the weight of blending α^* is performed on a held-out partition of the same dataset distribution as the training data. In the single-condition sub-datasets FD001 and FD003, with $\alpha^* = 1.0$ on the validation partition, the ensemble is no better than the standalone LSTM on the test set - indicating that the holdout optimization does not generalize in all sub-datasets. The optimization strategy based on cross-validation could yield a better and more generalizable set of blending weights, especially in the cases of the simpler sub-datasets in which the performance profile of the validation partition does not necessarily capture the performance of the test set.

6.3.5 No Feature-Level Interpretability

Although the proposed framework generates correct RUL predictions, it lacks any mechanism to explain which sensor channels or engineered features a certain prediction was made due to a certain engine at a certain cycle. Operation engineers do not only need accurate predictions but explainable predictions: when the system tells them that engine 47 is Critical and has a predicted RUL of 15 cycles, the engineer should know which sensors are indicating abnormal readings and what component subsystem is most apt to be the issue. Lack of feature-level attribution is a serious disconnects between the existing system and interpretability conditions of actual maintenance decision support.

6.3.6 Computational Resource Requirements

The LSTM branch training program needs a Tesla T4 with 15.6 GB of VRAM, which can be provided via cloud computing platforms but might not be available in resource-limited industrial edge deployment systems where inference needs to be performed near the engine sensors in real-time. The XGBoost component, which is computationally less demanding, however, still needs meaningful processing resources to be trained on the full C-MAPSS feature matrices. To fit the computational footprint into the limits of the available hardware, model compression, quantisation, or knowledge distillation would be required to deploy the model to embedded maintenance systems or on-board avionics health monitoring units.

6.4 SUGGESTIONS FOR FUTURE RESEARCH

The restrictions outlined in Section 6.3 directly point towards a list of specific and high energy directions of research in the future. All the suggestions below are based on a certain limitation and provide a technically clear direction of dealing with it.

6.4.1 Validation on Real-World Engine Data

The second most important action that this research can take is to validate the suggested framework with actual turbofan engine data outside the C-MAPSS simulation. More realistic experimental platforms, intermediate steps, are provided by the PHM 2008 Challenge data set that includes real accelerated degradation data of ball bearings and the PRONostia bearing data set. In case of turbofan, the specific validation would have to be conducted in collaboration with airline maintenance organizations or engine manufacturers to access real-life flight recorder information and corresponding maintenance logs. This validation would demonstrate the generalization of the framework Preprocessing pipeline, normalization

strategy and ensemble architecture to real sensor noise behavior, missing data behavior, and varying operating profile.

6.4.2 Bayesian Uncertainty Quantification

By adding uncertainty quantification to the RUL prediction pipeline, the operational utility of the framework would be greatly enhanced in helping to support the maintenance decision. There are two methods which are well established and are suitable in this application. It might be possible to introduce Monte Carlo Dropout, where dropout is applied during inference to obtain a distribution of prediction, and confidence intervals can be computed based on the distribution, without changing the LSTM architecture significantly.

Deep ensembles, which train many independently initialized LSTM models and average their predictions, are a more principled way to estimate uncertainty, but require more training time. Both of these would allow the maintenance status classification system to not only give a point prediction and status label, but risk-stratified confidence level, enabling maintenance engineers to give interventions the urgency they warrant.

6.4.3 Adaptive Meta-Learner Ensemble Strategy

Substituting the fixed scalar blending weight α with a condition-sensitive meta-learner is also a promising line of advancement to enhance ensemble performance, especially on the multi-condition sub-datasets. Instead, a meta-learner (which may be a small neural network, a gradient boosted classifier or a simple rule-based look-up) would be fed the current operating condition cluster assignment and an approximation of the stage of engine degradation, and would produce a dynamic blending weight α(c, d) which changes the relative contribution of each branch to the particular operational context at the inference time.

Such a strategy would transition the ensemble to a locally adaptive combination strategy, which has a globally optimal combination strategy, and which may be able to capture the different complementarity of the two branches at different stages of the engine lifecycle.

6.4.4 Cross-Validation for Robust Alpha Optimization

In any sub-data datasets in which the fixed scalar blending weight fails to cause evident ensemble benefit relative to the single-run LSTM i. e. FD001 and FD003, a cross-validation approach to the optimization of α can yield more resilient and generalisable blending weights. Instead of optimizing α on one held-out partition, a k-fold engine-level cross-validation would assess the ensemble performance using multiple held-out engine subsets, and give a cross-validated estimate of the optimal α that is less sensitive to the composition of the

validation partition. This would be useful especially in low-data-volume environments where the held-out partition might not reflect the entire test set distribution.

6.4.5 SHAP-Based Feature Attribution for Explainability

Incorporating SHAP (SHapley Additive exPlanations) value analysis into the framework would offer sensor-level attribution that elucidates what features influenced each engine to predict RUL individually. In the case of the XGBoost branch, SHAP values can be calculated efficiently with the TreeExplainer that generates the Shapley values of all 73 input features without approximations. In the case of the LSTM branch, the SHAP DeepExplainer or KernelExplainer might be used on the attention-weighted context vector to determine the most significant features and time steps to make a specific prediction.

This attribution capability would directly resolve the interpretability gap in Limitation 6.3.5 and would give maintenance engineers the sensor-level diagnostic data required to determine the particular component subsystem that has caused a predicted degradation event - bridging the gap between predictive model behavior and desired maintenance action.

6.4.6 Generalization to Other Industrial Domains

The hybrid framework suggested is not necessarily engine prognostics of the turbofan type. The architectural design, which is based on a temporal sequence model with a tabular feature model via an adaptive blending weight, and normalization using clusters of data when there is a variety of operating conditions, is generalizable to any industrial prognostics system in which run-to-failure data exists and there are a number of operating conditions. Applications domains Areas of candidate applications are wind turbine gearbox prognostics, where a multi-condition problem akin to FD002 and FD004 is posed by multiple wind speed regimes [22]; industrial bearing condition prognostics; pump and compressor health prognostics; and railway wheel condition prognostics. Applying the validation of the framework to these domains would make it a general and strong framework outside the aerospace environment.

6.4.7 Edge Deployment Optimization

To be practically applied to embedded maintenance monitoring systems or on-board avionics health management units, the computational footprint of the framework needs to be significantly decreased. Knowledge distillation - training a smaller student LSTM model to imitate the predictions of the full teacher LSTM may allow an order of magnitude reduction in the size of the model without a significant loss in predictive accuracy. The memory and inference time requirements can also be further minimized by using post-training

quantization which decreases the numeric accuracy of model weights to 8-bit integers. The compressed model could then be optimized using TensorRT or ONNX Runtime to run on the edge inference hardware with a GPU. These optimization measures would make the framework fit into the resource limits of the real-time engine health monitoring systems used in the field.

6.5 SUMMARY

This chapter has compiled the findings of the project in an organized report on contributions, findings, limitations, and future directions. Section 6.1 introduced the five main technical contributions of the framework under proposal: the hybrid LSTM-XGBoost ensemble with learned blending weight, the KMeans operating condition normalization strategy, the soft attention mechanism, the Huber Loss training process, and the three-tier maintenance status classification system: each of them was related to the respective research gaps they are addressing (Table 6.1).

In section 6.2, the essential five findings of the experimental evaluation were introduced. The most remarkable conclusion is that the best modeling strategy is identified based on the complexity of the operating condition of the dataset: the standalone LSTM with Attention is the best performing model on the single-condition sub-datasets FD001 and FD003, and the Hybrid Ensemble is the one that is consistently the most performing on the multi-condition sub-datasets FD002 and FD004 (Table 6.2). It was found that the LSTM consistently performs better than XGBoost as an independent model on all sub-datasets, and that the biased conservative prediction behaviour on multi-condition sub-datasets is operationally favourable to safety-critical maintenance scheduling.

Section 6.3 identified and discussed six study limitations, which included the use of simulated C-MAPSS data exclusively, uncertainty was not quantified, the scalar blending weight was fixed, the held-out partition alpha optimization, the inability to interpret features at the feature level, and the computational resource requirements to train (Table 6.3). Section 6.4 provided seven specific and technically detailed proposals to future research to overcome all these limitations: validation on real-world data, quantifying uncertainty with a Bayesian approach, adaptive meta-learner ensembles, alpha optimization through cross-validation, feature attribution with SHAP, generalization to other industrial sectors, and edge deployment optimization.The suggested hybrid LSTM-XGBoost ensemble model in the form of soft-

attention-adaptive-blending-weight is an academic, reproducible, and technically sound extension of the data-driven prognostics and health management domain. Its performance on NASA C-MAPSS benchmark, with a RMSE of 13.54 on FD003 and a R^2 of 0.8345 on FD002, makes it competitive in the published literature, and the modular structure and well-defined future directions offer it a solid basis to continue with research and development to real-world implementation in aviation predictive maintenance systems.

REFERENCES

All references are listed below in the order of their first appearance in the body of this report. Each reference has been cited at least once using the corresponding number in square brackets.

[1] C. Patil, S. Jadhav, A. Bardiya, A. Davande, and M. Raverkar, "Machine Learning-Based Predictive Maintenance of Industrial Machines," in Proceedings of the International Conference on Emerging Trends in Engineering and Technology, 2023.

[2] J. N. A. Malaiyappan, G. Krishnamoorthy, and S. Jangoan, "Predictive Maintenance using Machine Learning in Industrial IoT," in Proceedings of the International Conference on Intelligent Systems and Computing, 2024.

[3] N. Sharma, C. Chauhan, A. Dwivedi, and S. Dass, "A Review of Predictive Maintenance Methods for Electrical Machines Using Machine Learning Algorithms and Sensor Data," Journal of Engineering Research and Applications, 2025.

[4] W. Li and T. Li, "Comparison of Deep Learning Models for Predictive Maintenance in Industrial Manufacturing Systems," Applied Sciences, vol. 15, 2025.

[5] J. Garcia, L. Rios-Colque, A. Pena, and L. Rojas, "Condition Monitoring and Predictive Maintenance: An NLP-Assisted Review," Expert Systems with Applications, 2025.

[6] H. Singh, "IoT Based Predictive Maintenance in Industrial Machinery," International Journal of Engineering and Computer Science, 2025.

[7] A. Aziz and Sowmyashree P, "AI-Powered Predictive Maintenance System for Industrial Equipment," Journal of Artificial Intelligence and Machine Learning, 2025.

[8] A. K. S. Jardine, D. Lin, and D. Banjevic, "A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance," Mechanical Systems and Signal Processing, vol. 20, no. 7, pp. 1483–1510, 2006.

[9] H. Abouloifa and M. Bahaj, "Comparing Deep Learning and Fourier Series Models for Equipment Failure Prediction," Engineering Applications of Artificial Intelligence, 2025.

[10] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation," in Proceedings of the 1st International Conference on Prognostics and Health Management (PHM), Denver, CO, USA, 2008.

[11] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 2016, pp. 785–794.

[12] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[13] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in Proceedings of the 3rd International Conference on Learning Representations (ICLR), San Diego, CA, USA, 2015.

[14] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "LSTM Network for Remaining Useful Life Estimation," in Proceedings of the IEEE International Conference on Prognostics and Health Management (ICPHM), Dallas, TX, USA, 2017.

[15] X. Li, Q. Ding, and J. Sun, "Remaining Useful Life Estimation in Prognostics Using Deep Convolution Neural Networks," Reliability Engineering and System Safety, vol. 172, pp. 1–11, 2018.

[16] P. J. Huber, "Robust Estimation of a Location Parameter," Annals of Mathematical Statistics, vol. 35, no. 1, pp. 73–101, 1964.

[17] S. Deng and J. Zhou, "Prediction of Remaining Useful Life of Aero-engines Based on CNN-LSTM-Attention," International Journal of Computational Intelligence Systems, vol. 17, no. 232, 2024.

[18] H. Li, Z. Wang, and Z. Li, "An Enhanced CNN-LSTM Remaining Useful Life Prediction Model for Aircraft Engine with Attention Mechanism," PeerJ Computer Science, vol. 8, e1084, 2022.

[19] L. Wang, Z. Zhu, and X. Zhao, "Dynamic Predictive Maintenance Strategy for System Remaining Useful Life Prediction via Deep Learning Ensemble Method," Reliability Engineering and System Safety, vol. 245, 110012, 2024.

[20] H. Dehghan Shoorkand, M. Nourelfath, and A. Hajji, "A Hybrid CNN-LSTM Model for Joint Optimization of Production and Imperfect Predictive Maintenance Planning," Reliability Engineering and System Safety, vol. 241, 2024.

[21] S. Ye et al., "A Deep Learning-Based Prognostic Approach for Predicting Turbofan Engine Degradation and Remaining Useful Life," Scientific Reports, vol. 15, 2025.

[22] S. S. Shah, D. Tan, and S. C. Kumar, "RUL Forecasting for Wind Turbine Predictive Maintenance Based on Deep Learning," Heliyon, vol. 10, e39268, 2024.

[23] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," UN General Assembly, New York, 2015.

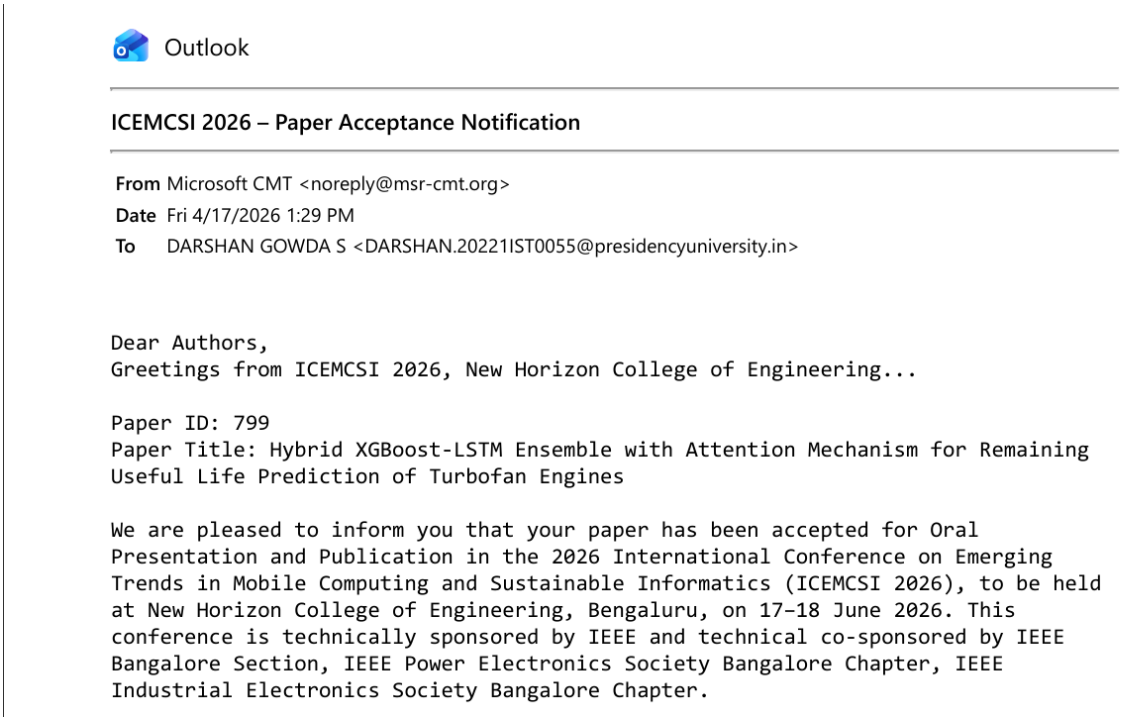
BASE PAPER

From References the mainly referred paper: [17] Deng, S. and Zhou, J. (2024) 'Remaining Useful Life of Aero-engines Prediction using CNN-LSTM-Attention', *International Journal of Computational Intelligence Systems*, vol. 17, no. 232.

The main basis of this study relates to the study conducted by Deng and Zhou [17], who united attention mechanisms and CNN-LSTM networks to solve the turbofan engine Remaining Useful Life prediction on the NASA C-MAPSS benchmark. Inspired by their work, we go further to replace the convolutional extraction branch with an XGBoost gradient boosted tree regressor, introducing a per-dataset adaptive blending weight (α) optimized by grid search to intelligently blend two complementary modelling strategies, and incorporating a KMeans-based operating condition normalization strategy that effectively separates flight regime shifts from genuine degradation signals in multi-condition operating environments.

APPENDIX A

A.1Proof of Research Paper Acceptance — ICEMCSI 2026



Outlook

ICEMCSI 2026 – Paper Acceptance Notification

From Microsoft CMT <noreply@msr-cmt.org>
Date Fri 4/17/2026 1:29 PM
To DARSHAN GOWDA S <DARSHAN.20221IST0055@presidencyuniversity.in>

Dear Authors,
Greetings from ICEMCSI 2026, New Horizon College of Engineering...

Paper ID: 799
Paper Title: Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines

We are pleased to inform you that your paper has been accepted for Oral Presentation and Publication in the 2026 International Conference on Emerging Trends in Mobile Computing and Sustainable Informatics (ICEMCSI 2026), to be held at New Horizon College of Engineering, Bengaluru, on 17–18 June 2026. This conference is technically sponsored by IEEE and technical co-sponsored by IEEE Bangalore Section, IEEE Power Electronics Society Bangalore Chapter, IEEE Industrial Electronics Society Bangalore Chapter.

A.1 Paper Acceptance Notification from ICEMCSI 2026 (IEEE)

The research work presented in this project has been accepted for Oral Presentation and Publication **at the** 2026 International Conference on Emerging Trends in Mobile Computing and Sustainable Informatics (ICEMCSI 2026)**, held at New Horizon College of Engineering, Bengaluru, on 17–18 June 2026. The conference is technically sponsored by** IEEE **and co-sponsored by IEEE Bangalore Section. The paper titled** "Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines" **(Paper ID: 799) has been successfully submitted as a camera-ready manuscript and, upon presentation, will be submitted for inclusion in the** IEEE Xplore Digital Library **(ISBN: 979-8-3315-5336-4), subject to meeting IEEE Xplore's scope and quality requirements.**

APPENDIX B

B.1Dataset Information and Access

NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) data can be found at the NASA Prognostics Data Repository as a publicly available dataset. It consists of four sub-datasets (FD001-FD004) of multivariate time-series sensor data of simulated turbofan engine run-to-failure paths.

Dataset Source: NASA Prognostics Data Repository

Access URL: **https://data.nasa.gov/dataset/C-MAPSS-Aircraft-Engine-Simulator-Data/xaut-bemq**

Dataset Loading Code Snippet

```
COLUMNS = ['engine_id','cycle','op1','op2','op3'] + [f's{i}' for i in
range(1,22)]

DROP_SENSORS = ['s1','s5','s6','s10','s16','s18','s19']

train_df = pd.read_csv(train_path, sep=r'\s+', header=None, names=COLUMNS)

train_df.drop(columns=DROP_SENSORS, inplace=True)

train_df['RUL'] = (train_df['max_cycle'] -
train_df['cycle']).clip(upper=125)
```

APPENDIX C

C.1 Code Snippet — LSTM with Attention

```
class Attention(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.attn = nn.Linear(hidden_size, 1)
    def forward(self, lstm_out):
        w = torch.softmax(self.attn(lstm_out), dim=1)
        return (w * lstm_out).sum(dim=1)

class LSTMModel(nn.Module):
    def __init__(self, input_size):
        super().__init__()
        self.lstm = nn.LSTM(input_size, 128, 2, batch_first=True,
dropout=0.2)
        self.attention = Attention(128)
        self.bn  = nn.BatchNorm1d(128)
        self.fc1 = nn.Linear(128, 64)
        self.fc2 = nn.Linear(64, 1)
```

C.2 Code Snippet — XGBoost Configuration

```
xgb_model = XGBRegressor(
    max_depth=3, learning_rate=0.03,
    subsample=0.6, colsample_bytree=0.6,
    reg_alpha=1.0, reg_lambda=5.0,
    early_stopping_rounds=50)
```

C.3 Code Snippet — Hybrid Ensemble Alpha Optimisation

```
res = minimize_scalar(
    lambda a: np.sqrt(np.mean(
        (hold_y - (a*pred_xgb + (1-a)*pred_lstm))**2)),
    bounds=(0, 1), method='bounded')
```

C.4 Data Augmentation

Implicit augmentation was done by sliding window construction. Each engine, with N operational cycles, was represented by N labelled training sequences of length 50, and zero-padded for engines with fewer than 50 cycles. This produced 124,909 training sequences in all the four sub-datasets without any artificial data.

Code Snippet — Sliding Window Sequence Construction

```
def make_sequences(df, feat_cols):
    X, y = [], []
    for eid in df['engine_id'].unique():
        edf = df[df['engine_id']==eid].reset_index(drop=True)
        for i in range(SEQUENCE_LENGTH, len(edf)):
            X.append(feats[i-SEQUENCE_LENGTH:i])
    return np.array(X, dtype=np.float32), np.array(y, dtype=np.float32)
```

C.5 Training Logs of all models Across all datasets.

```
── Training LSTM (FD001) ──
Epoch  10/100 | Train: 132.5995 | Val: 128.0320 | LR: 0.000976
Epoch  20/100 | Train: 125.6461 | Val: 146.3990 | LR: 0.000905
Epoch  30/100 | Train: 114.6406 | Val:  78.2695 | LR: 0.000796
Epoch  40/100 | Train:  73.1360 | Val:  54.7505 | LR: 0.000658
Epoch  50/100 | Train:  66.1938 | Val:  48.9690 | LR: 0.000505
Epoch  60/100 | Train:  61.7741 | Val:  45.6329 | LR: 0.000352
Epoch  70/100 | Train:  55.0289 | Val:  42.7205 | LR: 0.000214
Epoch  80/100 | Train:  47.8864 | Val:  33.1283 | LR: 0.000105
Epoch  90/100 | Train:  45.2150 | Val:  28.7462 | LR: 0.000034
Epoch 100/100 | Train:  44.1197 | Val:  27.9434 | LR: 0.000010
 Best Val Loss: 27.8944

── Training LSTM (FD002) ──
Epoch  10/120 | Train:  81.3762 | Val:  87.6493 | LR: 0.000983
Epoch  20/120 | Train:  70.1489 | Val:  68.7624 | LR: 0.000934
Epoch  30/120 | Train:  65.3800 | Val:  53.5924 | LR: 0.000855
Epoch  40/120 | Train:  60.7931 | Val:  66.7103 | LR: 0.000752
Epoch  50/120 | Train:  56.8055 | Val:  49.5156 | LR: 0.000633
Epoch  60/120 | Train:  52.1729 | Val:  43.2035 | LR: 0.000505
Epoch  70/120 | Train:  46.6065 | Val:  36.8028 | LR: 0.000377
Epoch  80/120 | Train:  38.4537 | Val:  28.4781 | LR: 0.000257
Epoch  90/120 | Train:  31.5683 | Val:  21.9551 | LR: 0.000155
Epoch 100/120 | Train:  27.0543 | Val:  17.5305 | LR: 0.000076
Epoch 110/120 | Train:  24.9712 | Val:  15.2575 | LR: 0.000027
Epoch 120/120 | Train:  24.2804 | Val:  14.0476 | LR: 0.000010
 Best Val Loss: 14.0476

── Training LSTM (FD003) ──
Epoch  10/100 | Train: 133.9864 | Val: 112.0010 | LR: 0.000976
Epoch  20/100 | Train:  71.0716 | Val:  48.2730 | LR: 0.000905
Epoch  30/100 | Train:  57.7990 | Val:  39.4686 | LR: 0.000796
Epoch  40/100 | Train:  54.1416 | Val:  38.2847 | LR: 0.000658
Epoch  50/100 | Train:  53.2027 | Val:  34.2316 | LR: 0.000505
Epoch  60/100 | Train:  47.8231 | Val:  31.5547 | LR: 0.000352
Epoch  70/100 | Train:  47.6997 | Val:  33.6621 | LR: 0.000214
Epoch  80/100 | Train:  44.0341 | Val:  29.5152 | LR: 0.000105
Epoch  90/100 | Train:  42.4878 | Val:  26.4156 | LR: 0.000034
Epoch 100/100 | Train:  41.4545 | Val:  25.9696 | LR: 0.000010
 Best Val Loss: 25.5083

── Training LSTM (FD004) ──
Epoch  10/120 | Train:  85.2125 | Val:  79.9760 | LR: 0.000983
Epoch  20/120 | Train:  66.7070 | Val:  57.8352 | LR: 0.000934
Epoch  30/120 | Train:  60.5387 | Val:  56.9063 | LR: 0.000855
Epoch  40/120 | Train:  55.9758 | Val:  48.2038 | LR: 0.000752
Epoch  50/120 | Train:  48.0462 | Val:  36.2227 | LR: 0.000633
Epoch  60/120 | Train:  39.9304 | Val:  32.7079 | LR: 0.000505
Epoch  70/120 | Train:  31.8271 | Val:  23.8346 | LR: 0.000377
Epoch  80/120 | Train:  28.6616 | Val:  16.2079 | LR: 0.000257
Epoch  90/120 | Train:  24.2907 | Val:  14.9935 | LR: 0.000155
Epoch 100/120 | Train:  21.7528 | Val:   9.1817 | LR: 0.000076
Epoch 110/120 | Train:  20.1314 | Val:   8.3983 | LR: 0.000027
Epoch 120/120 | Train:  19.6570 | Val:   8.5803 | LR: 0.000010
 Best Val Loss:  7.5848
```

Figure C.1 LSTM training console output showing epoch-wise training loss, validation loss, and learning rate for all four sub-datasets (FD001–FD004).

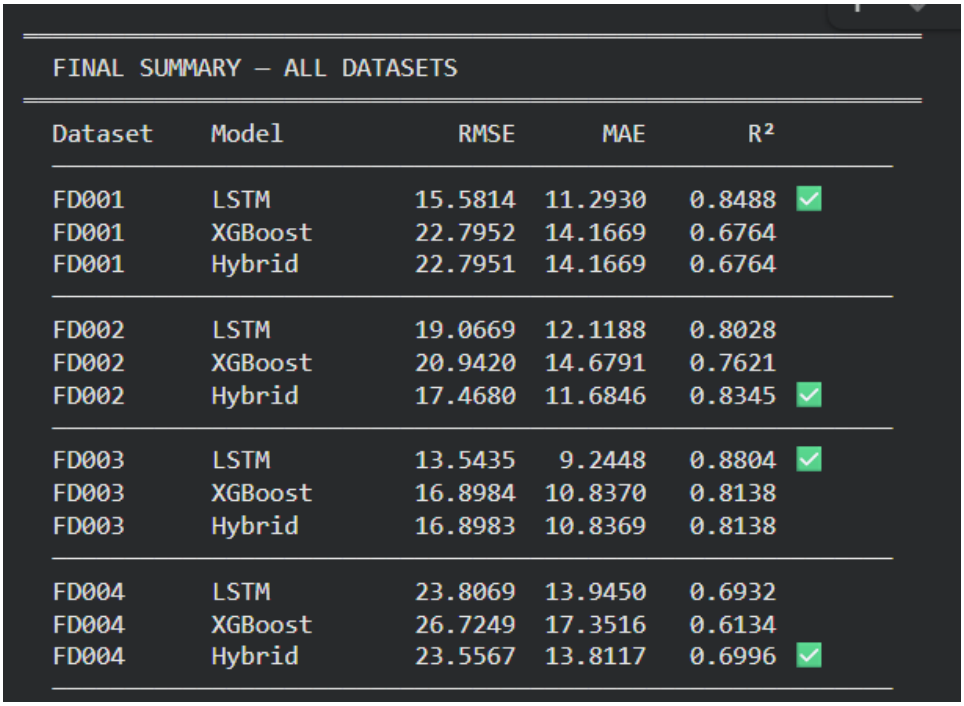
```
-- Training XGBoost (FD001) --
[0]     validation_0-rmse: 40.57328
[200]   validation_0-rmse:  8.28947
[400]   validation_0-rmse:  6.34164
[600]   validation_0-rmse:  5.20187
[800]   validation_0-rmse:  4.44016
[1000]  validation_0-rmse:  3.91174
[1200]  validation_0-rmse:  3.50120
[1400]  validation_0-rmse:  3.17656
[1499]  validation_0-rmse:  3.04557
[OK] Best XGB iteration: 1499

-- Training XGBoost (FD002) --
[0]     validation_0-rmse: 40.87351
[200]   validation_0-rmse: 16.69636
[400]   validation_0-rmse: 14.92457
[600]   validation_0-rmse: 13.57473
[800]   validation_0-rmse: 12.50170
[1000]  validation_0-rmse: 11.63301
[1200]  validation_0-rmse: 10.86784
[1400]  validation_0-rmse: 10.20528
[1600]  validation_0-rmse:  9.61649
[1800]  validation_0-rmse:  9.09847
[1999]  validation_0-rmse:  8.64813
[OK] Best XGB iteration: 1999

-- Training XGBoost (FD003) --
[0]     validation_0-rmse: 41.05751
[200]   validation_0-rmse:  8.22382
[400]   validation_0-rmse:  6.57197
[600]   validation_0-rmse:  5.50422
[800]   validation_0-rmse:  4.74744
[1000]  validation_0-rmse:  4.20627
[1200]  validation_0-rmse:  3.76718
[1400]  validation_0-rmse:  3.40712
[1499]  validation_0-rmse:  3.26039
[OK] Best XGB iteration: 1499

-- Training XGBoost (FD004) --
[0]     validation_0-rmse: 41.02180
[200]   validation_0-rmse: 15.85578
[400]   validation_0-rmse: 14.08022
[600]   validation_0-rmse: 12.87535
[800]   validation_0-rmse: 11.91752
[1000]  validation_0-rmse: 11.12294
[1200]  validation_0-rmse: 10.42004
[1400]  validation_0-rmse:  9.83714
[1600]  validation_0-rmse:  9.31271
[1800]  validation_0-rmse:  8.85124
[1999]  validation_0-rmse:  8.43667
[OK] Best XGB iteration: 1999
```

Figure C.2 XGBoost training console output showing validation RMSE at every 200th boosting round for all four sub-datasets.



FINAL SUMMARY — ALL DATASETS

Dataset	Model	RMSE	MAE	R^2	
FD001	LSTM	15.5814	11.2930	0.8488	✅
FD001	XGBoost	22.7952	14.1669	0.6764	
FD001	Hybrid	22.7951	14.1669	0.6764	
FD002	LSTM	19.0669	12.1188	0.8028	
FD002	XGBoost	20.9420	14.6791	0.7621	
FD002	Hybrid	17.4680	11.6846	0.8345	✅
FD003	LSTM	13.5435	9.2448	0.8804	✅
FD003	XGBoost	16.8984	10.8370	0.8138	
FD003	Hybrid	16.8983	10.8369	0.8138	
FD004	LSTM	23.8069	13.9450	0.6932	
FD004	XGBoost	26.7249	17.3516	0.6134	
FD004	Hybrid	23.5567	13.8117	0.6996	✅

Figure C.3 Console output showing the optimal α* per sub-dataset, holdout RMSE for all three model variants

APPENDIX D

D.1 Research Paper

Hybrid XGBoost-LSTM Ensemble with Attention Mechanism for Remaining Useful Life Prediction of Turbofan Engines

Darshan Gowda S
Dept. of Information Science and Technology, Presidency University, Bengaluru, India
DARSHAN.20221IST0055@presidencyuniversity.in

Chirag Gowda S V
Dept. of Information Science and Technology, Presidency University, Bengaluru, India
CHIRAG.20221IST0112@presidencyuniversity.in

Dr. Afroz Pasha
School of Computer Science and Engineering, Presidency University, Bengaluru, India
afrozpasha@presidencyuniversity.in

Chethan C
Dept. of Information Science and Technology, Presidency University, Bengaluru, India
CHETHAN.20221IST0069@presidencyuniversity.in

Abstract—Predictive maintenance of aircraft turbofan engines is a major issue in aerospace engineering whereby effective predictive maintenance of Remaining Useful Life (RUL) may be used to avoid disastrous failure and also to save on operational expenses. In other words, schemes presently exist largely on single-model frameworks of deep learning frameworks that learn temporal relations or classical machine learning frameworks that utilize statistical characteristics and fail to leverage the synergistic advantages of the two paradigms combined. This paper suggests a hybrid ensemble model in the form of an attention-based Long Short-Term Memory (LSTM) network and an Extreme Gradient Boosting (XGBoost) regressor that is used to predict RUL of the turbofan engines. The LSTM attention mechanism takes sensor sequences of 73-dimensional feature vectors (3 operating condition settings, 14 raw sensor readings and 4 statistical features × 14 sensors = 56 engineered features) through 50 cycles to learn how to degrade over time, whereas XGBoost predicts using the same engineered feature space as viewed statistically. A grid search is used to optimize a dataset-specific blending weight α on a separate validation partition to produce a weighted ensemble prediction. In order to address the problem of multi-condition engine operation in FD002 and FD004, KMeans clustering is used to determine six operating regimes and per-cluster Z-score normalization to de-couple sensor baseline shifts and actual degradation signals. LSTM training uses Huber Loss with $\delta = 10$ so that it is robust to outlier engines. The offered system is tested on the four sub-datasets of NASA C-MAPSS benchmark. Findings have shown that the most successful model depends on the complexity of the sub-datasets: standalone LSTM performs best on FD001 (RMSE = 15.58) and FD003 (RMSE = 13.54), whereas the hybrid ensemble performs better on the multi-condition sub-datasets FD002 (RMSE = 17.47, $R^2 = 0.83$) and FD004 (RMSE = 23.56, $R^2 = 0.70$).

Index Terms—Remaining Useful Life, Predictive Maintenance, LSTM, XGBoost, Hybrid Ensemble, Attention Mechanism, NASA C-MAPSS, Turbofan Engine Degradation, Huber Loss.

I. INTRODUCTION

The safety and reliability standards of the field of engineering are among the most rigid in aviation business. The key to any commercial airliner- the engine that converts fuel to thrust under extreme conditions of thermal and mechanical stress, flight after flight, cycle after cycle, is the turbofan engine. Such stress wears off major internal parts eventually. The High-Pressure Compressor stages and fan are particularly vulnerable and once these wear is not detected early enough, it leads to the loss of performance, components break down and in worst-case scenarios lives are lost [8].

The economic fact is also immediate. Engine maintenance is the largest percentage of all aviation MRO spending worldwide. Airlines that perform late maintenance can face unlikely grounding and aircraft failures. Airlines that book too early waste resources refurbishing parts that have a lot of life left in them. Both the results are unacceptable in the safety-critical and competitiveness industry. This is the question that was answered by Predictive Maintenance (PdM). Rather than fix a failed engine, or even replace components at a defined cost, PdM uses the sensor data already flowing from the engines to determine how much operational life each individual engine remains of its potential- its Remaining Useful Life (RUL). The maintenance can subsequently be planned at the most optimal time: late enough to allow the components to be utilized fully, yet not too late otherwise the risk of failure is a possibility [2], [6]. Recent developments in deep learning systems that predictive maintenance have further demonstrated the significance of combining model predictions and operational planning models [20].

RUL estimation is mathematically an estimation of the number of remaining number of active cycles before an engine exceeds a set failure threshold, given its sensor history to the current cycle. It is far more challenging than an average regression work. There exist numerous flight regimes (altitude, speed, throttle position) over which engines are operated and sensor baselines vary in a superficially degradation-like manner [10]. Raw sensor values are also distorted by measurement errors and atmospheric variability and the number of full run-to-failure trajectories to be used in training is limited in nature.

Over the last decade, machine learning has achieved actual development in this issue. Initially, random forests and support vector regression have performed well such that competitive RUL predictions could be obtained based on manually designed statistical features [1], [3]. The

Figure D.1 Research Paper

APPENDIX E

E.1 GitHub Repository and Streamlit Deployment

E.1.1 Main Project Repository:

https://github.com/darshan99009/IST_27-Capstone_Project

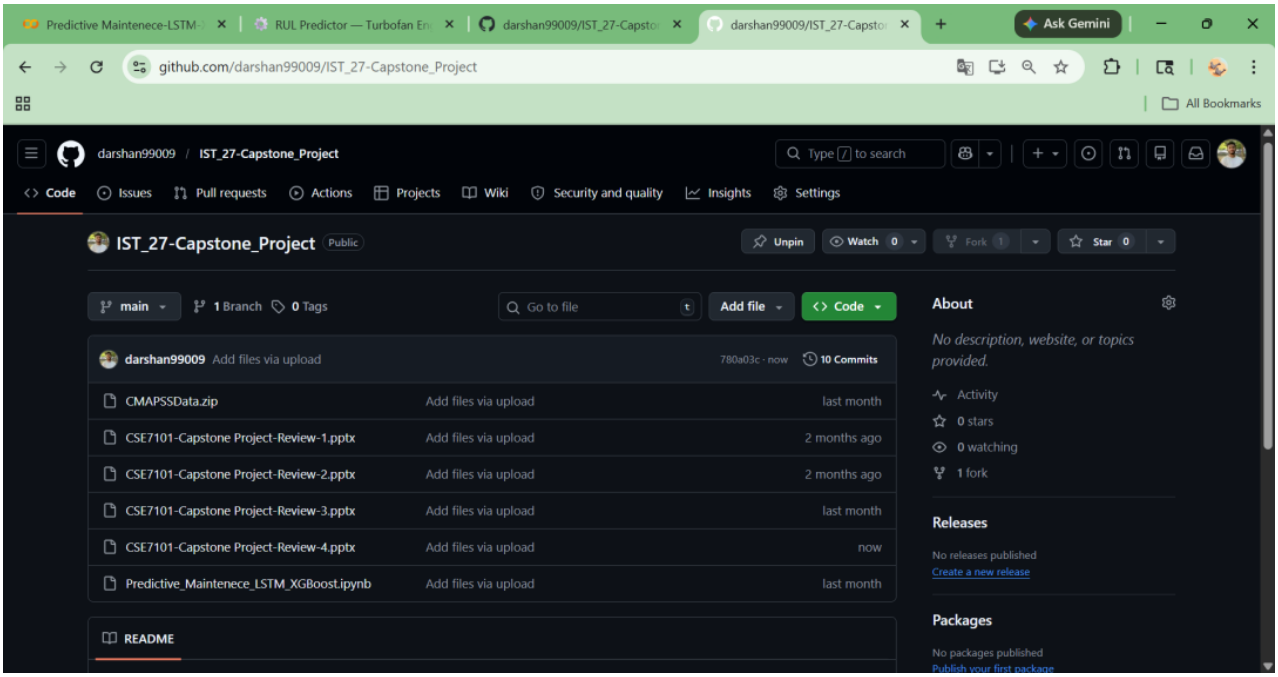


Figure E.1 Main Project Github Repository Containing All Project Files

E.2 Streamlit Deployment Repository:

https://github.com/darshan99009/predictive-main

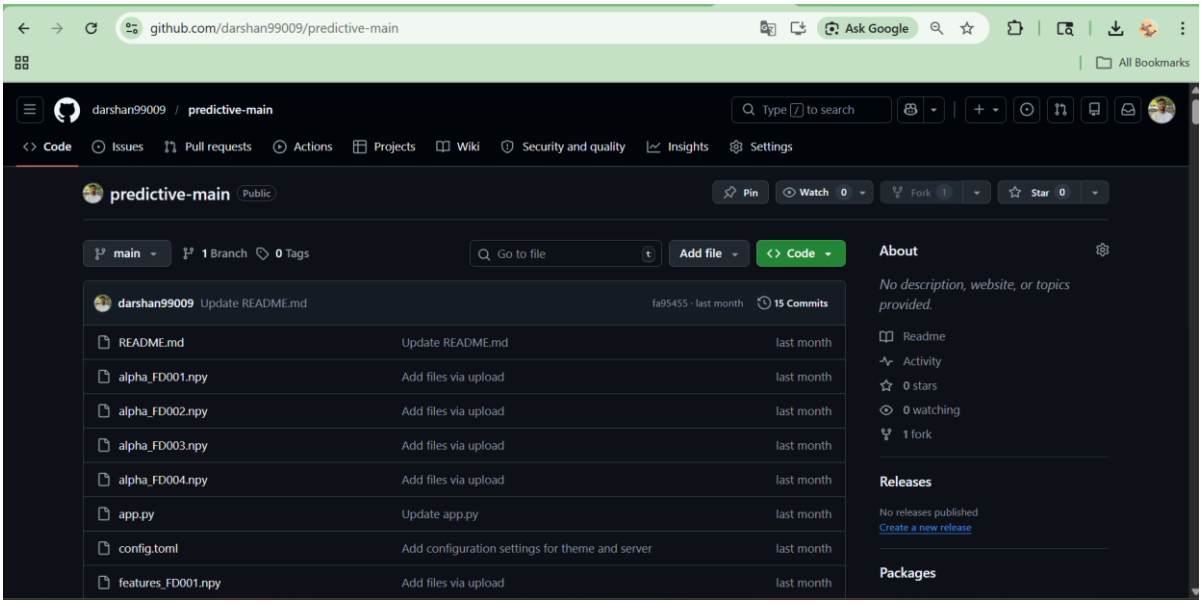


Figure E.2 Github Repository Connected to Streamlit

Live Streamlit Application:

https://predictive-main-6zqfnrdbtguflkfgf3vym3.streamlit.app/